Abstract

This dissertation investigates the security implications of cloud misconfigurations in Amazon Web Services (AWS), with a focus on Amazon S3 buckets and Elastic Compute Cloud (EC2) instances. Misconfigurations remain a leading cause of breaches in cloud environments, yet they are often overlooked despite extensive documentation of their risks.

A controlled testbed was created within the AWS Free Tier to replicate real world weaknesses, including publicly writable S3 buckets, insecure Apache2 configurations, and exposure of the legacy EC2 Instance Metadata Service (IMDSv1). Using tools such as AWS CLI, curl, and SSH, deliberate misconfigurations were introduced and exploited to demonstrate data leakage, privilege escalation, and the misuse of over privileged IAM roles. These scenarios were mapped to the OWASP Top Ten categories, specifically Broken Access Control and Security Misconfiguration.

The findings align with high profile breaches such as Accenture (2017) and Capital One (2019), showing that simple oversights can expose organisations to disproportionate risk. S3 misconfigurations resulted in direct public access to sensitive files, while IMDSv1 exploitation enabled the extraction of temporary credentials and potential lateral movement. Together, these vulnerabilities highlight the dangers of chaining attacks in cloud ecosystems.

Mitigation strategies were drawn from AWS security documentation, CIS benchmarks, and industry frameworks. Recommendations emphasise enforcing least privilege, mandating IMDSv2, enabling S3 Block Public Access, and adopting continuous monitoring with GuardDuty and CloudTrail.

The study acknowledges limitations, including reliance on Free Tier resources and manual testing, but contributes a reproducible methodology for exploring cloud security weaknesses. Future research should extend to multi account deployments, automated scanning, and adversary emulation at enterprise scale.

Contents

List of Abbreviations

Abbreviation	Full Form
ACL	Access Control List
AWS	Amazon Web Services
CLI	Command Line Interface
EC2	Elastic Compute Cloud
IAM	Identity and Access Management
IMDS	Instance Metadata Service
JSON	JavaScript Object Notation
MITM	Man-in-the-Middle
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PoC	Proof of Concept
S3	Simple Storage Service
SP	Special Publication
VPC	Virtual Private Cloud
ENISA	European Union Agency for Cybersecurity
GDPR	General Data Protection Regulation
IaC	Infrastructure as Code
MFA	Multi Factor Authentication
SIEM	Security Information and Event Management
WAF	Web Application Firewall
JSON	JavaScript Object Notation
SSRF	Server Side Request Forgery
CSF	Cybersecurity Framework

Chapter 1: Introduction

1.1 Background

The rise of cloud computing has reshaped how organizations deploy and manage their digital infrastructure. Its promise of scalability, flexibility, and lower costs has driven widespread adoption across industries. Among the leading cloud service providers, Amazon Web Services (AWS) holds a dominant position, offering a vast portfolio of services, including Amazon S3 (Simple Storage Service) and EC2 (Elastic Compute Cloud). However, as cloud environments become more configurable and complex, they also present new challenges particularly in the area of security.

Cloud misconfigurations have emerged as a major source of risk. Misconfigured S3 buckets exposing sensitive data or EC2 instances left open to the internet are not rare oversights they're consistent features in many high profile breaches (Liu, 2021). Despite AWS providing strong documentation and built in security tools, human error continues to account for a large portion of cloud related incidents (AWS, 2023a).

Recent breaches involving Capital One, Accenture, and public sector organizations underscore the severity of these risks (Chen, Xu, and Zhang, 2022). Issues like overly permissive access controls, weak IAM policies, and open EC2 ports have led to data leaks and service outages. This highlights a persistent gap: even well resourced teams are failing to implement secure configurations.

The problem is magnified by the growing complexity of cloud native architectures. The adoption of Infrastructure as Code (IaC), container orchestration, and microservices has created sprawling permission structures and dependencies. Every overlooked permission or exposed endpoint becomes a possible entry for attackers. A 2023 report by Palo Alto Networks found that 80% of cloud security incidents stem from misconfigurations not provider vulnerabilities making this one of the most pressing security concerns in cloud adoption today.

1.2 Problem Statement

Despite growing awareness and increasingly advanced cloud security tools, misconfigurations in AWS environments continue to result in data breaches and operational risks. There remains a clear need to understand how these misconfigurations occur, how they can be exploited, and most importantly how they can be prevented. Security guidelines alone are not enough; practical insight into exploitation methods and mitigation strategies is essential to closing this gap.

1.3 Research Aim

This dissertation aims to investigate how misconfigurations in AWS S3 buckets and EC2 instances can be identified and exploited from a penetration tester's perspective, and to propose effective mitigation techniques based on current security frameworks and real world practice.

1.4 Research Objectives

- To analyze common misconfiguration patterns in AWS S3 and EC2 services.
- To simulate real world attack scenarios in a controlled AWS environment using ethical penetration testing techniques.
- To propose practical mitigation strategies grounded in AWS security best practices and supported by industry frameworks such as the AWS Well Architected Framework and the CIS AWS Foundations Benchmark.
- To raise awareness among cloud administrators and cybersecurity professionals about the operational risks tied to poor configuration practices.

1.5 Research Scope and Methodology

This study focuses specifically on two widely used AWS services: S3 and EC2. S3 buckets are frequently misconfigured to allow public access or unauthorized write permissions. EC2 instances often suffer from open or unmonitored ports, weak SSH configurations, and excessive privileges through IAM roles. These services were selected for their critical role in cloud infrastructure and their recurring presence in breach case studies.

The research employs a mixed approach. A theoretical review of literature and security standards forms the basis of contextual understanding. Practical testing takes place in a controlled AWS Free Tier environment, using tools such as AWS CLI and curl to simulate common attacker techniques (Owens and Newman, 2020). The study does not involve testing on real world production systems or advanced adversarial tactics. Instead, it replicates foundational misconfiguration exploits in a controlled environment, while drawing on real world breach case studies to contextualize the practical risks and consequences.

1.6 Ethical Considerations

All testing is conducted exclusively within personal AWS accounts using isolated, non production environments. No third party data, systems, or networks are accessed at any stage of the research. The study complies with the UK Cyber Security Council’s Code of Ethics and has received ethical approval from the university. All data generated during testing is anonymized and securely stored in accordance with academic and legal standards.

1.7 Significance of the Study

This research serves multiple stakeholders. For cybersecurity practitioners, it provides a technical guide to identifying and mitigating common AWS misconfigurations. For organizations and cloud administrators, it highlights how small missteps can lead to largescale breaches. For academic researchers, it contributes to the growing literature on cloud-specific vulnerabilities and the human factors that shape them.

Chapter 2: Literature Review

2.1 Literature Review Methodology

This literature review adopts a narrative, desk based approach to explore the security risks of cloud misconfigurations particularly within Amazon Web Services (AWS) environments such as S3 and EC2. To ensure relevance, the review integrates peer reviewed academic literature, industry breach reports, official cloud security frameworks, and tool specific documentation. Sources were critically selected to inform both the theoretical and practical components of the research.

Inclusion and Exclusion Criteria

Included sources:

- Peer reviewed journal articles (2015–2024) focused on cloud security or misconfiguration
- Industry whitepapers and threat intelligence reports (e.g. IBM X-Force, Palo Alto Networks, Cloud Security Alliance)
- AWS official documentation and technical blogs
- Real world breach analyses involving AWS misconfigurations (e.g. Capital One, Accenture)
- Technical documentation and literature discussing security tools commonly used in AWS misconfiguration detection and exploitation (e.g. ScoutSuite, Pacu, Prowler)

Excluded sources:

- Non English publications
- General news articles without technical analysis
- Opinion based blogs, discussion forums, or YouTube content without citations
- Studies unrelated to AWS or cloud infrastructure misconfiguration

Search Strategy and Data Collection

The literature was sourced through a combination of academic search engines (e.g. Google Scholar), curated industry publications, and official documentation portals provided by AWS and leading cybersecurity vendors. Supplementary insights were gathered from security research blogs and guided secondary sources. The following keywords and Boolean combinations were used to guide the search:

- “AWS S3 misconfiguration”
- “EC2 security vulnerability”

- “Cloud metadata service exploitation”
- “AWS penetration testing tools”
- “CIS AWS Benchmark” AND “cloud compliance”
- “Well-Architected Framework” AND “cloud security”
- “Privilege escalation in AWS”
- “Cloud misconfiguration breach case study”

Search filters were applied to prioritize English language publications between 2015 and 2024 that provided technical depth and direct relevance to AWS security.

Thematic Organization and Analysis

Relevant sources were reviewed and grouped thematically to align with the chapter structure: cloud misconfiguration types, breach case studies, detection and exploitation tools, and cloud security frameworks. References were manually organized and cited consistently using the university's required Harvard referencing style. This structured literature base supports the practical testing methodology detailed in Chapter 3.

2.2 Overview of Cloud Misconfigurations

Cloud computing has rapidly become the backbone of modern digital infrastructure, enabling scalable, on demand access to computing resources. However, as cloud adoption has grown, so too has the number of security incidents resulting from misconfigurations. Unlike traditional software vulnerabilities that exploit vulnerabilities in code, misconfigurations often stem from human error incorrect permissions, insecure defaults, or overlooked policy settings that leave critical systems exposed to threat actors (Liu, 2021).

Numerous reports and studies have emphasized that misconfigurations represent a primary threat vector in cloud environments. According to a 2023 IBM X-Force Threat Intelligence report, cloud misconfigurations were responsible for over 40% of publicly reported cloud security breaches that year. Similarly, the Cloud Security Alliance (2022) identified misconfiguration and inadequate change control as the top cause of data breaches in cloud-native deployments.

This type of risk is particularly prevalent in Infrastructure as a Service (IaaS) environments such as Amazon Web Services (AWS), where users are responsible for managing their own configuration and access controls. While AWS provides the underlying secure infrastructure, the shared responsibility model requires customers to configure their resources properly a task many organizations continue to struggle with.

2.3 Common Misconfigurations in AWS S3 and EC2

Among AWS's suite of services, S3 (Simple Storage Service) and EC2 (Elastic Compute Cloud) are two of the most widely used and most frequently misconfigured offerings. S3, which stores vast amounts of unstructured data, is commonly misconfigured to allow public access, leading to unintentional data exposure. According to a 2022 report by Rapid7, over 30,000 S3 buckets were publicly accessible at the time of scanning, many of which contained sensitive or proprietary data.

Typical S3 misconfigurations include:

- Setting bucket policies to Allow: * (public access)
- Failing to enable encryption at rest or in transit
- Misconfigured CORS settings allowing broad cross origin access

EC2, meanwhile, provides virtual server instances and is susceptible to a different range of configuration errors. One of the most common is the use of overly permissive security groups virtual firewalls that control inbound and outbound traffic. Allowing SSH access from 0.0.0.0/0 (i.e. the entire internet) or leaving administrative ports like 3389 (RDP) open without multi factor authentication can quickly become an entry point for brute force attacks and malware deployment (Owens and Newman, 2020).

In addition, IAM role mismanagement such as assigning EC2 instances with permissions far beyond what is necessary (violating the principle of least privilege) can allow attackers to escalate privileges or access S3 buckets and other services through metadata endpoints.

2.4 Real World Case Studies

To better understand the impact of misconfigurations, it is important to examine real world breaches. These high-profile incidents highlight not just technical gaps, but also failures in operational oversight and cloud governance.

Capital One (2019)

The Capital One data breach in July 2019 stands as one of the most impactful examples of cloud misconfiguration being exploited at scale. A former Amazon Web Services employee, Paige Thompson, leveraged a server-side request forgery (SSRF) vulnerability in a web application firewall (WAF) connected to a Capital One EC2 instance (Newman, 2019). By exploiting the SSRF flaw, the attacker was able to query the EC2 instance metadata service (http://169.254.169.254/latest/meta-data/) and extract temporary AWS credentials assigned to the instance via an IAM role.

These credentials granted access to over 700 S3 buckets, enabling the attacker to exfiltrate approximately 106 million customer records, including names, addresses, credit scores, and social security numbers. The root cause of the breach was not a vulnerability in AWS itself, but rather a misconfigured IAM role that granted the EC2 instance excessive permissions, violating the principle of least privilege. The incident cost Capital One more than $190

million in regulatory fines, settlements, and remediation efforts, and it highlighted the urgent need for fine grained IAM policy control, WAF hardening, and metadata protection strategies such as IMDSv2 (AWS, 2023b).

Accenture (2021)

In 2021, security researchers at UpGuard discovered that four Amazon S3 buckets belonging to Accenture a leading global IT consultancy were publicly accessible due to misconfigured bucket permissions (UpGuard, 2021). The exposed data included API documentation, customer credentials, software source code, and internal IT architecture files. The exposed credentials, in some cases, could have granted access to client environments or internal DevOps pipelines, posing significant supply chain risks.

Although there was no confirmed exploitation, the incident drew attention to the visibility gap in cloud resource configuration, especially in environments with large teams and decentralized DevOps practices. Notably, Accenture had passed recent audits for cloud compliance, suggesting that compliance frameworks may lag behind emerging operational realities. This breach demonstrated that even organizations with deep cloud expertise are not immune to the risks of inconsistent S3 access control policies and misaligned organizational oversight.

U.S. Department of Defense (DoD) Exposure (2023)

In an alarming case of public sector cloud misconfiguration, over 3 terabytes of internal military emails managed by the U.S. Department of Defense were inadvertently exposed through an unsecured Microsoft Azure Blob Storage server (TechCrunch, 2023). The storage was configured for anonymous public access, lacking both authentication and encryption. The emails contained personnel files, sensitive deployment records, and internal directives posing substantial national security implications.

While not hosted on AWS, the architectural and procedural errors mirrored those frequently observed in AWS S3 misconfiguration cases, making it a relevant comparative example. The incident emphasized the cross platform ubiquity of misconfiguration risks and called into question the adequacy of cloud governance in even the most security sensitive environments. Analysts suggested that the likely cause was a misconfigured Infrastructure as Code (IaC) deployment, once again highlighting the fragility of cloud infrastructure when automation is not paired with rigorous validation pipelines.

Toyota (2022)

Toyota’s 2022 cloud incident involved a misconfigured S3 bucket that allowed unauthenticated public read access. Security researchers from Group IB discovered that the exposed bucket contained source code, internal configuration files, and credentials for

Toyota's internal cloud services and CI/CD pipelines (Group IB, 2022). The documents also included hardcoded API keys and environment variables, which could have allowed an attacker to pivot into live production systems.

This incident is a textbook case of how a seemingly minor configuration oversight can have far reaching consequences. Although Toyota acted swiftly to revoke the leaked credentials and secure the bucket, the fact that source code and access tokens were exposed to the public internet without detection raises questions about the efficacy of their internal cloud auditing tools and DevSecOps practices.

Key Themes and Implications

These case studies demonstrate recurring patterns in cloud misconfiguration incidents:

- Overly permissive IAM policies and neglected access controls remain a critical vulnerabilities, especially when used with EC2.
- Public access to cloud storage (S3 or similar) often stems from default settings, insufficient testing, or lack of review in deployment pipelines.
- Lack of visibility into cloud configurations across large, distributed teams contributes to persistent risk.
- Automation tools and CI/CD pipelines can unintentionally propagate misconfigurations at scale when guardrails are not enforced.
- Credential leakage through metadata services or exposed config files amplifies the impact of basic misconfigurations.

Collectively, these incidents reinforce the core argument of this dissertation: that cloud misconfigurations are not edge cases, but endemic vulnerabilities that require technical, procedural, and educational interventions to resolve.

2.5 Tools and Techniques in Detection & Exploitation

Detecting and exploiting misconfigurations in AWS environments requires a combination of manual inspection, command line interaction, and automated assessment tools. These tools are integral not only to offensive security professionals such as penetration testers and ethical hackers but also to cloud security architects and incident response teams seeking to proactively identify and remediate misconfigured cloud assets.

Manual and Command Line Techniques

A foundational utility in this domain is the AWS Command Line Interface (CLI), which allows authenticated users to interact programmatically with AWS resources. Using the CLI, a penetration tester can list S3 buckets (aws s3 ls), inspect permissions (aws s3api get-bucket-acl), or enumerate EC2 instance metadata and associated IAM roles (aws ec2 describe-instances). These commands offer granular visibility into resource exposure, and when combined with access credentials, allow for controlled testing of privilege boundaries (AWS, 2023a).

The curl command line tool is another staple in penetration testing, especially during exploitation of EC2 instances. One of the most notorious targets in cloud exploitation is the Instance Metadata Service (IMDS), accessible from within EC2 at the link local IP address http://169.254.169.254/latest/meta-data/. When EC2 instances are configured with over-privileged IAM roles, curl can be used to extract temporary security credentials from the metadata service. These credentials often include access key IDs, secret access keys, and session tokens, which attackers can use to impersonate the instance's role across other AWS services. This exact technique was employed in the Capital One breach, where a server side request forgery (SSRF) vulnerability enabled an attacker to access the metadata endpoint and retrieve IAM credentials (Newman, 2019).

In response to such incidents, AWS introduced IMDSv2, which enforces session based access tokens and HTTP PUT requests for additional protection. However, legacy instances may still rely on IMDSv1, making them attractive targets for lateral movement and privilege escalation in internal testing scenarios (AWS, 2023c).

Automated Discovery Tools

To scale the discovery of misconfigurations across large AWS environments, practitioners often use open source auditing tools like ScoutSuite, Prowler, and CloudMapper. These tools perform multi service audits, checking for risky configurations across IAM, EC2, S3, Lambda, RDS, and other AWS services.

- ScoutSuite by NCC Group is a multi cloud security auditing tool that collects data via APIs and produces comprehensive HTML reports. It identifies insecure IAM policies, publicly exposed S3 buckets, permissive security groups, and other high risk configurations (NCC Group, 2022).
- Prowler, developed by Toni de la Fuente, performs checks aligned with the CIS AWS Foundations Benchmark and can integrate with AWS Security Hub for centralized alerting. It supports both real time scanning and scheduled assessments, making it valuable for continuous compliance monitoring (Prowler, 2023).
- CloudMapper, originally developed by Duo Security, visualizes AWS architecture and flags suspicious configurations, such as internet facing EC2 instances or unused access keys.

These tools help bridge the gap between governance frameworks and practical enforcement, allowing security teams to benchmark their cloud posture against industry standards and best practices.

Exploitation and Simulation Frameworks

Beyond discovery, certain tools focus on post exploitation simulation replicating how a threat actor might chain misconfigurations to escalate privileges or access sensitive resources. A leading tool in this category is Pacu, developed by Rhino Security Labs. Pacu is an AWS-specific exploitation framework designed for red teams and offensive security assessments.

Pacu modules can automate attacks such as:

- Privilege escalation through IAM role chaining
- Enumeration of trust relationships
- Accessing S3 buckets via compromised credentials
- Modifying EC2 user data for persistence
- Executing Lambda based lateral movement (Rhino Security Labs, 2023)

Its modular structure allows testers to tailor attacks to specific misconfigurations observed during reconnaissance, providing a practical means of demonstrating risk to stakeholders in penetration testing reports.

Chained Exploitation Techniques

It is essential to understand that cloud misconfigurations rarely operate in isolation. Attackers typically exploit a chain of vulnerabilities, moving from initial foothold to lateral access or data exfiltration. For instance, an attacker might:

1. Discover a publicly readable S3 bucket
2. Extract an internal source code repository containing hardcoded AWS credentials
3. Use those credentials to assume a misconfigured IAM role
4. Query EC2 metadata from within a Lambda function to escalate further
5. Deploy backdoors or extract sensitive data from other services like RDS or DynamoDB

Such multi step exploitation sequences emphasize the interconnected nature of cloud services and the importance of defense in depth strategies. Automated tools may flag individual issues, but it is the contextual analysis by human testers that reveals the true severity and exploitability of a misconfiguration (Rhino Security Labs, 2023).

2.6 Cloud Security Frameworks and Guidelines

As misconfigurations continue to account for a significant proportion of cloud related security incidents, organizations increasingly rely on standardized frameworks and guidelines

to build and maintain secure cloud infrastructures. These frameworks offer structured approaches to risk assessment, control implementation, and continuous improvement. While no single framework can guarantee absolute security, their adoption provides a baseline for operational discipline, technical hygiene, and regulatory compliance in cloud environments such as AWS (Cloud Security Alliance, 2022; AWS, 2023b).

AWS Well Architected Framework

The AWS Well Architected Framework is one of the most widely adopted internal security and design blueprints within the AWS ecosystem. It comprises six pillars Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability each designed to guide architects and engineers in building secure, resilient, and efficient applications. The Security Pillar, in particular, focuses on five core areas: identity and access management, detection, infrastructure protection, data protection, and incident response (AWS, 2023b).

Key security recommendations include:

- Enforcing least privilege access via IAM roles and policies
- Activating CloudTrail logging to capture API activity across regions
- Using AWS Config to continuously audit configurations
- Automating responses to incidents using AWS Lambda and Amazon EventBridge
- Segmenting resources using VPCs and security groups to reduce attack surface

While the Well Architected Framework provides extensive guidance for AWS native security, its implementation depends heavily on organizational maturity and DevSecOps discipline. Without automated enforcement and routine architectural reviews, teams may drift from these best practices over time.

CIS AWS Foundations Benchmark

The Center for Internet Security (CIS) offers another widely respected set of recommendations: the CIS AWS Foundations Benchmark, which is a prescriptive standard designed to help organizations implement foundational cloud security controls. Now in version 1.5, the benchmark provides over 40 security checks across four main categories: Identity and Access Management (IAM), Logging, Monitoring, and Networking (CIS, 2023).

Some key controls include:

- Ensuring MFA is enabled for all root accounts
- Avoiding the use of root accounts for everyday operations

- Enabling S3 bucket versioning and logging
- Disabling public IP addresses on EC2 instances by default
- Enabling VPC flow logs to monitor traffic entering and leaving subnets

Unlike the broader Well Architected Framework, the CIS Benchmark is designed to be auditable and is often integrated into compliance tools such as Prowler, Security Hub, and AWS Config Rules. Organizations can use it as a compliance baseline during audits, penetration tests, or post incident reviews.

NIST Cybersecurity Framework (CSF)

Beyond AWS specific tools, the National Institute of Standards and Technology (NIST) offers a more general framework for cybersecurity risk management. The NIST Cybersecurity Framework (CSF) consists of five high level functions Identify, Protect, Detect, Respond, and Recover which serve as an operational lifecycle for managing security in dynamic IT environments, including cloud (NIST, 2022).

The CSF encourages organizations to:

- Identify assets and vulnerabilities using risk assessments
- Protect them using access control, data encryption, and segmentation
- Detect anomalies through logging and security analytics
- Respond with well defined incident handling protocols
- Recover from events using backups, business continuity plans, and forensic investigation

While the CSF is not AWS specific, it can be adapted to cloud environments using mappings provided by AWS itself. For example, AWS offers a NIST 800-53 Quick Start template that deploys infrastructure aligned with NIST controls using AWS CloudFormation. This helps organizations particularly in regulated industries like healthcare and government align their cloud security posture with federal or international expectations.

Limitations and Compliance Blind Spots

Although these frameworks offer a structured approach to security, they are not infallible. Organizations such as Accenture and Toyota were found to be technically compliant with many controls but still exposed sensitive data due to misconfigurations that fell outside the scope of standard checks (UpGuard, 2021; Group-IB, 2022).

This suggests that compliance does not equate to security. Compliance frameworks often focus on checking static configurations or specific conditions but fail to account for dynamic threats, human error, or toolchain vulnerabilities. For example, if developers accidentally hardcode AWS credentials in public GitHub repositories, this may not trigger a CIS violation, but it still constitutes a severe security risk.

Moreover, many frameworks assume a relatively static environment, while modern DevOps workflows rapidly deploy and update infrastructure using Infrastructure as Code (IaC). If proper validation and enforcement are not applied at the pipeline level, misconfigurations can be introduced and propagated at scale without violating any existing benchmarks.

2.7 Research Gap and Contribution

The growing body of literature on cloud security reflects the increasing prioritization of cybersecurity in digital infrastructure. Numerous academic and industry studies have examined cloud vulnerabilities, data protection frameworks, and compliance strategies. However, a review of the existing research reveals a significant gap between policy driven discourse and hands on security experimentation, particularly in the context of misconfiguration based vulnerabilities within AWS environments.

Underrepresentation of Practical Exploitation Studies

Academic literature has often focused on governance models, compliance auditing, and risk quantification, with limited attention given to the practical mechanics of misconfiguration exploitation. For instance, Liu (2021) provided a broad review of misconfiguration types and their business impacts but did not include any simulated testing environments or reproducible workflows. Similarly, Fang et al. (2023) focused on identifying misconfiguration patterns in Infrastructure as Code (IaC) but did not explore how those configurations could be actively exploited under ethical conditions. As such, most studies stop short of demonstrating how attackers can leverage these flaws in real world scenarios using publicly available tools.

This leaves a knowledge gap for cybersecurity practitioners who are expected to defend against these exact threats. Understanding how a misconfiguration occurs is valuable but understanding how it is discovered, chained, and weaponized provides a more complete perspective, critical for preparing effective mitigations.

Lack of Reproducible, Ethical Testing Models

Additionally, most industry whitepapers and breach analyses (e.g. from AWS, Palo Alto Networks, or IBM X-Force) highlight misconfigurations as a leading threat vector but lack reproducible testing methodologies. They typically present findings retrospectively, often in the form of statistics or high level summaries, with little transparency about tools, commands, or testing flows used. This makes it difficult for learners, educators, and professionals to replicate or validate those scenarios for research or training purposes.

Moreover, there is limited academic work combining AWS Free Tier environments with structured ethical penetration testing. While some tutorials and blogs exist within the InfoSec community, few peer reviewed or university backed studies walk through the controlled exploitation of real AWS vulnerabilities, especially not with clear documentation of ethical approval, methodology, toolchain, and results. The absence of formalized test labs in the academic literature leaves a void in cybersecurity education and limits the development of simulation based curricula that reflect real world attacker behaviour.

Contribution of This Dissertation

This dissertation addresses the above gaps by conducting a practical, ethically grounded investigation of AWS S3 and EC2 misconfigurations, blending academic research with hands on penetration testing. The project adheres to a structured methodology aligned with cybersecurity research standards and includes risk assessments, ethical review, and step by step documentation. The core contributions are outlined below:

1. A Structured Methodology for Misconfiguration Simulation and Documentation

This contribution presents a repeatable, ethically designed lab methodology for simulating AWS misconfigurations. Using Free Tier resources, the study deploys vulnerable S3 buckets and EC2 instances to replicate real world issues such as public access, weak security groups, and exposed metadata. Each scenario is tested and documented in a controlled environment to provide practical insights into the risks and remediation of cloud misconfigurations.

2. Real World Case Mapping

By mapping each simulated misconfiguration to actual breach case studies such as Capital One (2019), Toyota (2022), and Accenture (2021) this research bridges theoretical exploration with empirical evidence. These mappings provide context for why specific vulnerabilities matter, how they have been exploited historically, and what the consequences were for affected organizations (Newman, 2019; UpGuard, 2021; Group-IB, 2022).

3. Practical Mitigation Strategies

The dissertation provides mitigation strategies grounded in both the AWS Well Architected Framework and the CIS AWS Foundations Benchmark. Recommendations include enforcing least privilege through IAM policies, disabling public S3 access by default, using AWS Config rules for drift detection, and restricting EC2 metadata access via IMDSv2. These suggestions are not only theory based but validated through field testing in the dissertation's own lab environment.

4. Toolchain Transparency and Reproducibility

Another key contribution is the explicit use and explanation of readily available security tools such as aws cli, and curl. Each tool is introduced with its purpose, mode of operation, and context of use. This transparency allows future researchers to replicate tests, extend them into more advanced adversarial simulations, or use them as part of cybersecurity teaching modules.

5. Bridging Theory and Practice

By integrating academic theory, industry frameworks, and penetration testing exercises, this dissertation offers a hybrid research model. It serves both academic and practical objectives adding depth to the literature on misconfiguration threats while also equipping readers with reproducible examples and test strategies they can apply in real world scenarios.

Conclusion

In summary, while cloud misconfigurations are widely acknowledged as a critical security risk, the literature has yet to fully integrate technical exploitation, ethical validation, and applied testing models into a cohesive research framework. This dissertation contributes a structured, reproducible, and academically sound approach to closing that gap, thereby enhancing understanding of how seemingly simple configuration errors can lead to severe security breaches in modern cloud environments.

Chapter 3: Methodology

3.1 Research Approach

This dissertation adopts a qualitative experimental research design, combining library based theoretical research with hands on penetration testing in a controlled cloud environment. The central aim is to investigate how common misconfigurations in AWS S3 buckets and EC2 web applications can be exploited and mitigated, using a structured, ethical, and reproducible methodology.

Unlike purely observational or survey based research, this project engages directly with cloud infrastructure through the AWS Free Tier. It follows the principles of experimental cybersecurity research, in which hypotheses about security misconfigurations and their consequences are tested through active simulation. This method supports in depth technical investigation and enables a practical understanding of how attackers might identify and exploit vulnerabilities introduced by improper configuration.

The study is grounded in a qualitative framework because it emphasizes the interpretation of testing outcomes such as the consequences of accessing EC2 metadata, or the scope of exposure in a public S3 bucket over statistical generalization. Qualitative methods are particularly suited to this project because they allow the researcher to document and analyze complex attack paths, tool behaviours, misconfiguration effects, and ethical implications in a nuanced, contextual manner.

To support systematic execution and documentation, the project incorporates an agile-inspired iterative process.

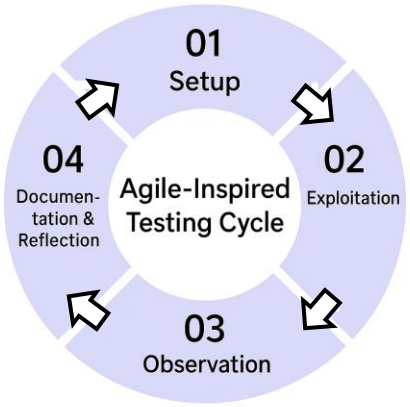


Figure 1: Agile-Inspired Testing Cycle

This diagram illustrates the iterative testing approach adopted in this study, comprising setup, exploitation, observation, and documentation. Each test cycle builds on previous findings, supporting systematic exploration of AWS misconfigurations.

Each testing cycle comprising setup, exploitation, observation, and logging is treated as a mini sprint. This model promotes flexibility and encourages reflection after each test scenario, improving the rigour of the methodology and ensuring the project remains adaptable to emerging findings.

Key characteristics of the research approach include:

Ethical experimentation within the AWS Free Tier

All infrastructure deployed and tested in this project resides within the researcher own AWS account, limited to Free Tier resources. This ensures full administrative control over all services and avoids unintended interference with third party systems or production environments. The infrastructure includes:

- S3 buckets with intentionally weakened access policies
- EC2 instances with deliberately misconfigured security groups and IAM roles

No real user data or third party cloud accounts are involved. Ethical approval was obtained through the university's review process, and a detailed risk assessment was completed prior to testing.

Minimal toolchain for accessibility and transparency

The testing environment intentionally relies on two lightweight, widely available tools:

- AWS Command Line Interface (CLI): Used to list, modify, and inspect AWS resources such as S3 permissions, EC2 metadata, and IAM policies
- curl: Used to simulate HTTP requests, particularly to EC2 metadata endpoints, mimicking common exploitation techniques like server side request forgery (SSRF) and role based credential theft

This minimalist approach was chosen to emphasize **reproducibility and accessibility**. Researchers, students, and practitioners can follow the same steps without needing complex or proprietary tools, making the methodology suitable for academic replication, teaching, or inclusion in lab based curricula.

Transferability and repeatability

The research design supports transferability to similar contexts, such as Azure or GCP misconfiguration testing, by documenting the entire process clearly. The AWS focused methodology including CLI commands, misconfiguration templates, test cases, and screenshots will be included in the appendices to promote repeatability.

The methodology also includes:

- Command line outputs were manually recorded during testing, and key inputs/outputs were captured via screenshots. While automated session logging (e.g., set -x) was not used, all commands were tested multiple times for consistency and documented immediately to ensure traceability. Screenshot documentation of each vulnerability before and after exploitation
- Validation of test results through repeated execution in separate AWS regions or resource sets

Alignment with industry practices and academic theory

Finally, the research approach bridges industry frameworks (e.g. AWS Well Architected Framework, CIS AWS Benchmark) with academic literature on misconfigurations, attacker behaviour, and ethical hacking. This hybrid model strengthens the relevance of the findings and supports the dual goals of:

- Contributing to academic discourse through a systematic review of cloud security practices
- Providing practical guidance for cloud engineers, penetration testers, and IT security managers

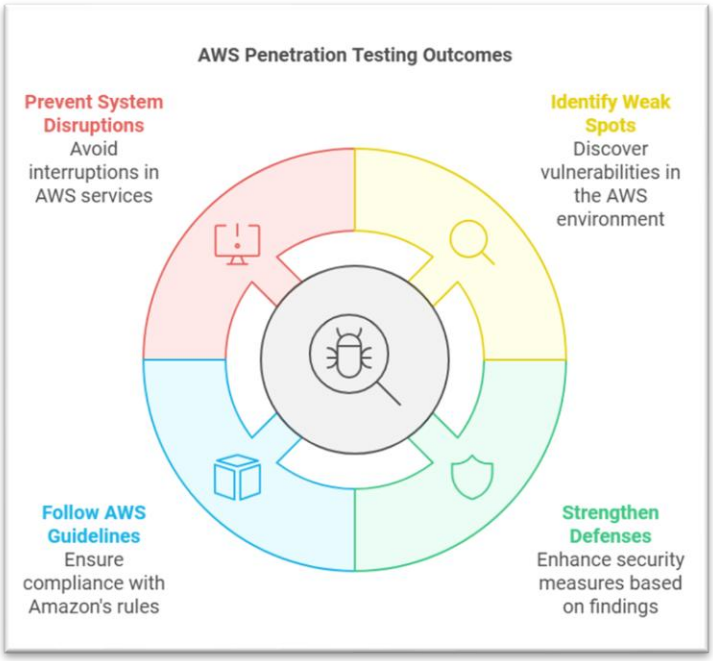
In summary, this research approach reflects a deliberate balance between technical execution and academic discipline. By combining ethical cloud penetration testing with a qualitative analytical lens, the project offers both real world insight and scholarly contribution into one of the most pervasive risks in cloud computing: AWS misconfigurations.

3.2 Testing Workflow and Lab Design

The research methodology is built around a structured, five phase workflow designed to simulate and analyze AWS misconfiguration vulnerabilities. This workflow draws upon established principles from penetration testing methodologies, including the OWASP Testing Guide (OWASP, 2023), NIST SP 800-115 (Scarfone et al., 2008), and AWS's penetration testing policy (AWS, 2023b), while adapting them to an academic and ethical testing context.

Each phase is logically sequenced to reflect a typical attacker's progression from reconnaissance, to exploitation, to post exploitation analysis and remediation. The process is iterative and allows for feedback loops; if findings in Phase 4 reveal unexpected behaviours, the environment is reconfigured and retested to validate insights and maintain methodological integrity.

A visual representation of this workflow is provided below:



This diagram outlines the general flow from setup to exploitation and reporting, aligning with the structure used in this project.

Phase 1: Permission and Scope Definition

The first phase focuses on ethical and procedural preparation. Before any testing activities are conducted, the scope of the project is clearly defined and formal ethical approval is obtained from the university's ethics board. The scope is limited to the my own AWS Free Tier account, ensuring complete ownership of infrastructure and eliminating the risk of impacting third party services.

During this phase:

- A written risk assessment is completed and submitted.
- All misconfigurations to be introduced are documented in advance.
- Testing objectives, tools, and attack scenarios are reviewed in relation to AWS's Acceptable Use Policy and Penetration Testing Policy.
- Monitoring and logging mechanisms are configured to ensure traceability of actions.

The outcome is a defined boundary within which the entire project operates, preserving academic integrity and compliance with cloud usage policies.

Phase 2: Environment Setup

Once ethical groundwork is complete, the next phase involves configuring the AWS lab environment. This is done using a combination of AWS Management Console and AWS CLI, allowing for scripting, automation, and precise replication.

Key elements of the lab include:

- S3 Buckets: Created with relaxed access controls such as “public-read” permissions or overly permissive bucket policies ("Principal": "*"). Encryption is left disabled to simulate a common misconfiguration.
- EC2 Instances: Deployed with public IP addresses and security groups allowing unrestricted inbound access on ports such as 22 (SSH) or 80 (HTTP). IAM roles are intentionally assigned with excessive permissions (e.g., AmazonS3FullAccess).
- Network Configuration: Resources were deployed within the default VPC environment provided by AWS Free Tier. No custom subnetting or network ACL modifications were implemented, aligning with the project’s goal of maintaining a minimal, reproducible test environment.

This setup mirrors the conditions under which many real world misconfigurations are discovered either due to poor security practices, oversight, or DevOps speed prioritization over compliance.

Phase 3: Attack Surface Mapping

After the environment is in place, the attack surface is enumerated using AWS CLI and basic network reconnaissance tools. This phase replicates what a legitimate penetration tester or malicious actor might perform during reconnaissance.

Activities include:

- Listing all S3 buckets with aws s3 ls and inspecting their access policies using aws s3api get-bucket-acl and get-bucket-policy.
- Describing EC2 instances using aws ec2 describe instances to uncover public IPs, security group configurations, and assigned IAM roles.

- Identifying IAM roles and attached policies to detect privilege escalation vectors.
- Using AWS CLI's sts get caller identity to confirm role assumption contexts.

This information is then analyzed to determine the path of least resistance, which forms the basis for the next phase: targeted exploitation.

Phase 4: Vulnerability Assessment and Exploitation

In this core testing phase, the researcher performs simulated exploitation of the configured misconfigurations. The goal is not to cause damage, but to validate how a misconfiguration could realistically be abused in a real world attack.

Examples of tests include:

- Using **curl** from within an EC2 instance to access the Instance Metadata Service (http://169.254.169.254/latest/meta-data/) and retrieve IAM credentials from the /iam/security-credentials/ path.
- Using those credentials to assume elevated roles and gain access to restricted S3 buckets (aws s3 cp or aws s3 ls).
- Attempting to upload or delete files from insecure S3 buckets to simulate data tampering.

Each test is documented with:

- Command line inputs and outputs
- Screenshots of successful access or privilege escalation
- Comments explaining what went wrong (e.g., "no bucket policy in place", "instance allowed full role assumption")

Phase 5: Reporting and Remediation

The final phase involves synthesizing all findings into actionable insights. Misconfigurations are logged using a consistent vulnerability report format that includes:

- Affected resource
- Vulnerability description
- Proof-of-concept command
- Security impact
- Recommended remediation

Although the findings were not formally mapped to the AWS Well-Architected Security Pillar or the CIS AWS Foundations Benchmark during testing, the observed vulnerabilities reflect known violations of their key principles such as lack of least privilege, improper logging, and insecure resource exposure. These frameworks are referenced in the discussion and recommendations to contextualize the findings.

Chaining Misconfigurations for Impact Analysis

A key emphasis in this workflow is on chaining multiple misconfigurations. For example:

1. An EC2 instance with an open port allows access to metadata.
2. Metadata leaks credentials for a privileged IAM role.
3. That role has permission to list or modify S3 buckets.
4. A public S3 bucket contains sensitive files, which can be accessed or modified.

This simulation demonstrates how minor oversights in isolation may seem harmless, but in combination, can enable full compromise of cloud resources. Understanding these chains is critical for defensive architecture planning.

3.3 Tools and Data Collection

3.3.1 Tools

The practical component of this research relies on a minimal, transparent toolset designed to maximize reproducibility while minimizing dependency on third party frameworks. The tools used are standard across AWS environments and do not require elevated privileges or additional installation beyond the AWS CLI.

- AWS Command Line Interface (CLI): The AWS CLI was the primary tool used for provisioning resources, enumerating configurations, and testing access permissions. Commands such as aws s3api get-bucket-acl, aws s3 ls, aws ec2 describe-instances, and aws sts get-caller-identity were employed to examine the state and exposure of cloud assets, including S3 buckets and EC2 instances. This allowed the researcher to interact directly with the AWS API in a lightweight, scriptable manner (Amazon Web Services, 2023).
- curl: This command line tool was essential for testing EC2 Instance Metadata Service (IMDS) accessibility. Requests were made to http://169.254.169.254/latest/meta-data/ and its associated credential paths to simulate role based credential exposure. Only IMDSv1 was tested, as it remains the default in many legacy deployments and allowed for unauthenticated metadata access during exploitation scenarios (Mitchell, 2021).
- Screenshot and Logging Tools: Testing outcomes were documented using screenshots taken at critical points during exploitation (e.g., after successful credential retrieval, or confirmation of S3 access). Additionally, command line outputs were manually recorded into structured documentation. While session based logging such as set -x

was not employed, outputs were validated by repeating commands and cross-referencing screenshot evidence.

No third party penetration testing tools, scanners, or exploitation frameworks (e.g., Pacu, ScoutSuite, Prowler) were used. The study intentionally limited the toolset to core AWS-native tools to ensure clarity, accessibility, and relevance for educational or low budget research environments (Cloud Security Alliance, 2022).

3.3.2 Data Collection

During the penetration testing process, a range of data points was collected to ensure comprehensive analysis and traceability. These include:

• Command Inputs and Outputs: Every AWS CLI and curl command used during testing was recorded along with its terminal output to verify whether the exploit was successful, denied, or partially successful. These outputs were essential for validating access paths and observing permission boundaries (AWS CLI User Guide, 2023).

• S3 Bucket and EC2 Configuration Details: Detailed information about resource configurations including S3 bucket policies, access control lists (ACLs), encryption settings, and EC2 security group rules was extracted and analyzed to correlate misconfigurations with observed behaviours (AWS Security Blog, 2019).

• IAM Role and Policy Details: The permissions attached to EC2 instances via IAM roles were reviewed to assess their adherence to the principle of least privilege. Special attention was paid to roles such as AmazonS3FullAccess, which allowed access to sensitive bucket contents (AWS IAM Best Practices, 2023).

• curl Request and Response Logs: Requests made to metadata endpoints and the structure of the returned JSON data (e.g., access key IDs, tokens) were captured to demonstrate potential for credential leakage (F5 Labs, 2025).

• Screenshots: For each exploit scenario, one or more screenshots were captured showing the commands used and their corresponding output. These images were stored in a clearly labelled and time stamped format within test-specific folders to support analysis and verification (ENISA, 2009).

3.4 Ethical and Compliance Measures

The ethical dimension of cybersecurity research particularly when it involves simulating attack behaviours in real world platforms requires careful planning, scope limitation, and institutional oversightThis study strictly adheres to both academic and cloud service provider standards to ensure responsible and compliant testing practices (UK Cyber Security Council, 2021).

Controlled Scope and Infrastructure Ownership

All testing was conducted exclusively within the researcher's personal AWS Free Tier account, ensuring full administrative control and eliminating the risk of unauthorized access to third party systems or customer environments. No shared production environments, public facing services, or externally hosted platforms were involved. By using only intentionally deployed resources, the research respects the principle of target ownership (EC-Council, 2021). A foundational standard in ethical hacking and penetration testing.

The infrastructure included:

- Custom S3 buckets and EC2 instances with deliberately weakened configurations
- IAM roles created solely for testing purposes
- Default VPC environments to simulate common cloud deployment scenarios

This self contained lab model ensured no impact beyond the researcher’s own cloud tenancy.

Institutional Ethical Approval

Prior to testing, the project was submitted for ethical review in accordance with the university’s guidelines for postgraduate research. The submission included:

- A detailed risk assessment
- Scope boundaries for technical activities
- A list of AWS services and test scenarios
- Data management and safety protocols

Formal ethical approval was granted, confirming that the project complies with the institution's expectations for responsible cybersecurity experimentation. All testing was conducted post approval, and any changes to the scope or tooling were documented accordingly.

Prohibition of Unauthorized Scanning or Exploitation

No scanning, enumeration, or exploitation of external IP ranges, domains, or cloud environments was performed. The study also explicitly avoided the following:

- Any form of testing on third party resources
- Accessing or manipulating live user data
- Probing shared resources or multi tenant cloud infrastructure

This complies with both academic ethical standards and cloud service usage agreements, preserving the legal and reputational integrity of the project.

Compliance with AWS Security and Penetration Testing Policies

The study followed AWS's Acceptable Use Policy and Shared Responsibility Model, which distinguish between AWS's obligations (e.g., physical infrastructure security) and customer responsibilities (e.g., IAM policies, network configuration). Since the testing was conducted on self owned infrastructure, no formal penetration testing approval was required per AWS's updated policy, internal testing of customer owned environments is allowed without pre-approval as long as certain conditions are met (AWS, 2023).

Data Privacy and Storage

No personal, sensitive, or third party data was accessed, stored, or processed during the study. All test outputs (e.g., command logs, screenshots, IAM credential samples) were generated using dummy data or system generated metadata from within the test environment. Files were stored locally in encrypted folders and labelled for auditability. No data was uploaded to external platforms, and no telemetry or behavioural data was sent to AWS or third party services.

3.5 Validation and Reproducibility

Ensuring the accuracy, consistency, and replicability of findings is critical in cybersecurity research, especially when simulating real world vulnerabilities (Jansen and Grance, 2011; ENISA, 2021). Several measures were implemented in this study to validate the results and support reproducibility.

Validation of Testing Results

To ensure reliability, each misconfiguration test was conducted in a controlled AWS Free Tier environment using consistent deployment steps and command sequences. While the tests were executed once per scenario, careful attention was paid to validating outputs through command line results and screenshots. This approach confirmed that each misconfiguration produced the expected vulnerability behaviour under the conditions tested. The results consistently reproduced the same misconfiguration behaviours across regions, strengthening internal validity. This repetition confirmed that vulnerabilities were rooted in configuration settings rather than transient AWS performance or network variables (Jansen and Grance, 2011).

Additionally, the findings were benchmarked against real world case studies such as the Capital One breach (2019) and the Toyota S3 exposure (2022). These comparisons helped validate that the misconfigurations tested in this dissertation are not just hypothetical flaws but reflective of actual incidents with significant impact. This alignment enhances the ecological validity of the research, demonstrating its relevance to real world cloud security failures (ENISA, 2021).

Reproducibility for Academic and Professional Use

The testing methodology including infrastructure setup, CLI commands, misconfiguration parameters, and exploit steps is documented in a stepby step lab guide included in the appendix. This guide enables other researchers, educators**,** or security practitioners to replicate the lab environment using only an AWS Free Tier account and basic command line tools.

Each test case is mapped to a discrete folder structure containing:

- CLI command logs
- Screenshots of output/results
- Observational notes
- Reference to the misconfiguration being demonstrated

This structured approach supports academic reproducibility, allowing future cybersecurity students or instructors to reuse the lab design as a teaching or testing framework.

3.6 Limitations of the Approach

While the research methodology is robust for its intended scope, it is important to acknowledge its limitations:

Scope Limitation

The project focused solely on AWS S3 and EC2 services, chosen due to their widespread use and frequent appearance in cloud breach reports (Rapid7, 2022). Other critical AWS services such as IAM in depth, Lambda, CloudTrail, or VPC routing were excluded due to time and resource constraints. Similarly, multi cloud environments (e.g., Azure, GCP) were outside the scope of this dissertation.

Tool Simplicity

Only AWS CLI and curl were used for testing to maximize transparency and ensure accessibility. While this minimal toolset is sufficient for demonstrating key misconfiguration risks, more advanced tools such as ScoutSuite, Pacu, or Burp Suite could uncover deeper privilege escalation paths, API misuse, or chained vulnerabilities not detected through basic enumeration alone (De La Fuente, 2023).

Environmental Constraints

All testing was performed within the AWS Free Tier, which imposes limits on available instance types, storage capacity, and network configurations. As a result, the lab environment lacks the scale and complexity of a real production deployment, which may involve autoscaling, multi tier architecture, continuous integration pipelines, and advanced identity federation. This limits the external validity of findings to larger scale or enterprise AWS deployments (AWS, 2024).

3.7 Chapter Summary

This chapter has presented a structured and ethically sound methodology for investigating misconfigurations in AWS S3 and EC2 services through a hands on penetration testing approach. The research design combines theoretical awareness, technical execution, and academic integrity to examine how commonly overlooked cloud configurations can lead to serious security vulnerabilities.

The methodology rests on four key pillars:

- A Clearly Defined Scope: The study is limited to two of the most widely used AWS services S3 and EC2 selected due to their prevalence in cloud environments and frequent appearance in real world breach reports. All testing is confined to the researcher's own AWS Free Tier account, eliminating any risk to third party systems or data.
- A Five-Phase Testing Workflow: The methodology follows a logical progression from ethical setup and environment deployment to exploitation and reporting. Each stage Permission and Scope, Environment Setup, Attack Surface Mapping, Exploitation, and Reporting ensures that the research remains systematic, reproducible, and aligned with professional cybersecurity practices.
- A Minimal but Effective Toolset: The research relies solely on the AWS Command Line Interface (CLI) and curl, tools that are freely available and widely used in both academic and professional contexts. This minimalist toolchain reinforces transparency, reproducibility, and accessibility important qualities for research that may be reused by students, educators, or entry level security practitioners.
- Ethical Safeguards and Compliance Alignment: Prior to testing, the project received formal university ethical approval. All activities were conducted in accordance with AWS's Shared Responsibility Model and Acceptable Use Policy. The study avoids scanning, probing, or interacting with any system or service outside the researcher's control, maintaining full compliance with institutional and industry standards.

Although tests were not repeated across multiple regions, outcomes were carefully validated through consistent CLI outputs and observational documentation. Additionally, findings were contextualized against high-profile case studies such as Capital One and Toyota, underscoring the real world relevance of the vulnerabilities examined.

All test scenarios are documented in detail including command usage, screenshots, and observed outcomes and are organized in a structured file system with supplementary notes. A simplified lab guide is provided in the appendix to support reproducibility and adaptation in educational or professional environments.

Chapter 4: Implementation

4.1 Introduction

This chapter outlines the design and deployment of the experimental environment created to simulate cloud misconfigurations within Amazon Web Services (AWS). The purpose of this implementation was to replicate real world weaknesses in Amazon S3 buckets and EC2 instances, thereby providing a controlled setting in which penetration testing could be conducted. By intentionally creating insecure configurations, the study aimed to investigate how such weaknesses manifest, how they can be exploited, and what mitigations are most effective in preventing them.

4.2 Implementation Methodology

The implementation of the testing environment followed a structured methodology designed to ensure reproducibility, ethical integrity, and alignment with recognised cloud security frameworks. The process was divided into four stages: design, deployment, misconfiguration, and validation. Each stage was informed by best practice guidelines from AWS, the CIS AWS Foundations Benchmark, and vulnerability frameworks such as the OWASP Top Ten (OWASP, 2021).

Design Stage

The first stage involved defining the scope and architecture of the lab. Amazon S3 and EC2 were selected as the core services due to their prevalence in real world cloud deployments and their frequent appearance in breach investigations. The architecture was intentionally kept simple a default VPC in the us-east-1 region to reflect a typical small to medium enterprise deployment, while remaining cost effective under the AWS Free Tier.

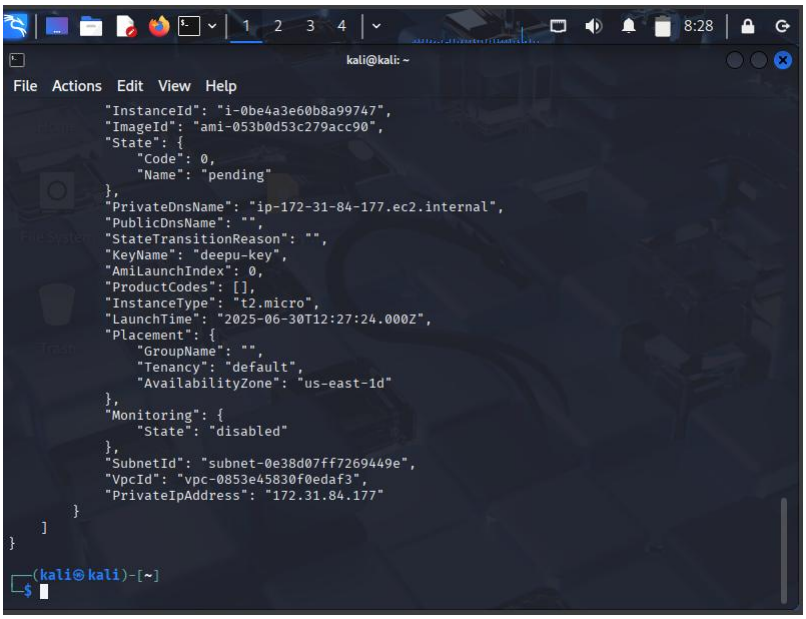


Figure 4.1: Launching an EC2 instance using AWS CLI with t2.micro configuration

Ethical considerations were embedded into this stage by ensuring no production systems, personal data, or third party resources were included in the design.

Deployment Stage

In the second stage, resources were provisioned using the AWS Management Console and AWS CLI. The console provided intuitive configuration management, while the CLI enabled verification of resource states and repeatability of commands.

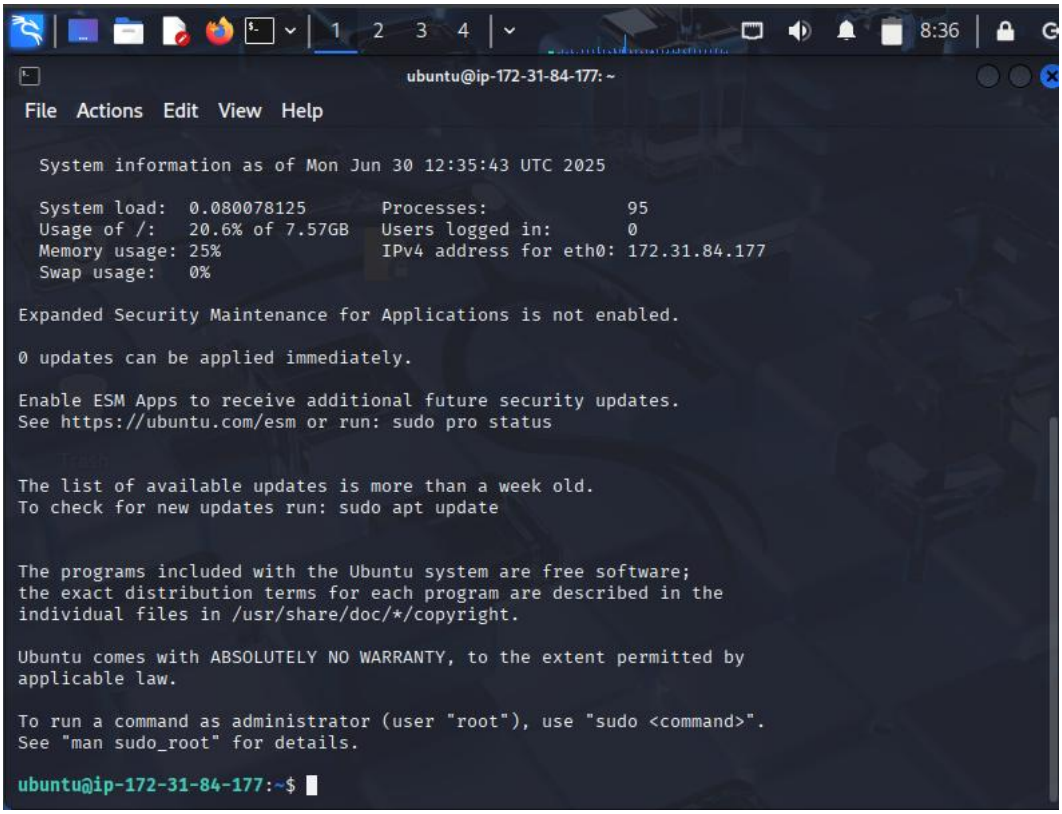


Figure 4.2: Accessing the deployed EC2 instance via SSH and verifying system details

A Kali Linux virtual machine served as the testing host, chosen for its integration of penetration testing utilities such as curl, nmap, and SSH clients. This ensured consistency between the implementation environment and the subsequent testing activities.

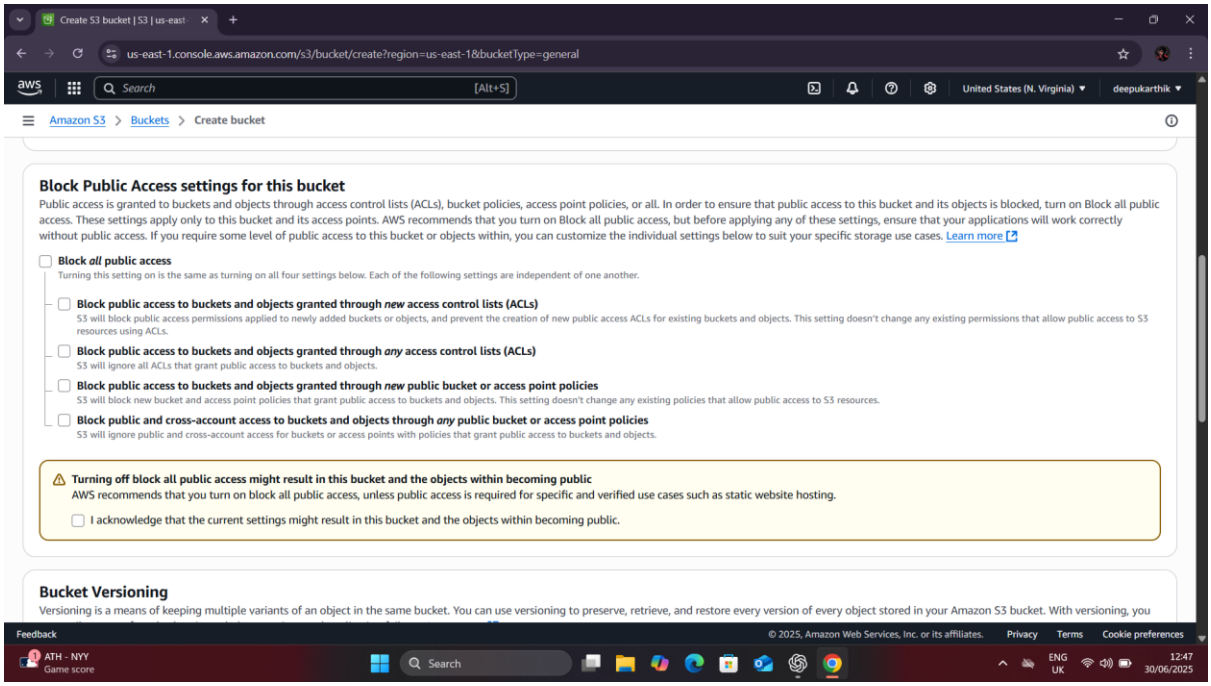


Figure 4.3: S3 bucket creation with Block Public Access disabled

Misconfiguration Stage

The third stage involved deliberately introducing insecure configurations in line with vulnerabilities frequently associated with cloud breaches. Examples included disabling S3 Block Public Access, assigning excessive IAM role permissions, and exposing the EC2 Instance Metadata Service (IMDSv1). These misconfigurations were selected not only because they have been exploited in real world cases such as the Capital One breach, but also because they map directly to categories within the OWASP Top Ten, particularly Broken Access Control (A01) and Security Misconfiguration (A05*)*. This ensured the environment reflected both academic and industry relevant vulnerabilities.

Validation Stage

The final stage consisted of validating that the environment replicated the intended weaknesses. Dummy files containing simulated sensitive information were uploaded to S3 and EC2 servers to confirm public accessibility. Metadata queries were tested on EC2 instances to ensure IMDSv1 was exposed. At each point, screenshots and logs were captured to support transparency and reproducibility. This validation ensured that the subsequent penetration tests could be conducted against a controlled and predictable baseline.

4.3 Design of AWS Environment

The design of the experimental environment was intended to replicate common patterns of cloud deployment while embedding deliberate security weaknesses for later testing. The architecture was built entirely within the AWS Free Tier to ensure cost effectiveness, ethical compliance, and reproducibility. All resources were deployed in the us-east-1 region within the default Virtual Private Cloud (VPC). This choice simplified configuration by relying on

AWS's preconfigured routing and subnet structure, while still maintaining realistic conditions under which vulnerabilities could be demonstrated.

Two core services formed the basis of the environment: Amazon S3 for object storage and Amazon EC2 for compute resources. Both were selected because they represent the foundation of most AWS workloads and have repeatedly been the focus of security incidents in industry case studies. For example, the Capital One breach in 2019 exploited an EC2 metadata service misconfiguration, while Accenture's 2017 exposure involved publicly accessible S3 buckets.

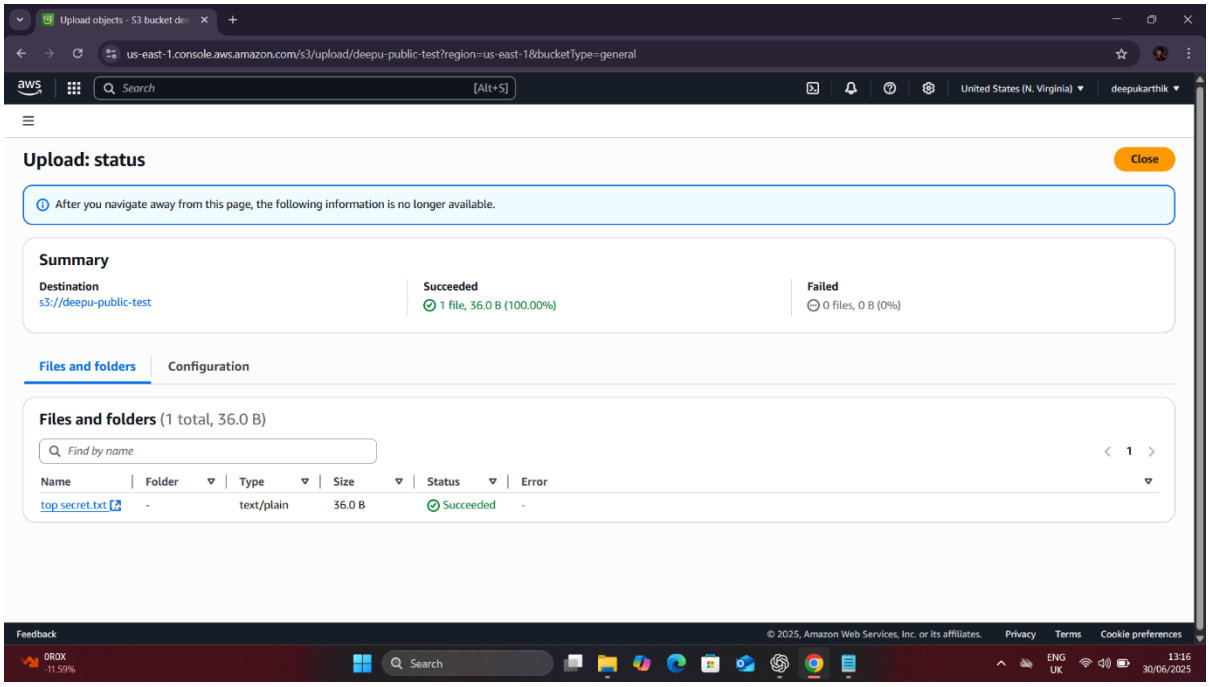


Figure 4.4: Uploading a dummy file (top secret.txt) into misconfigured S3 bucket

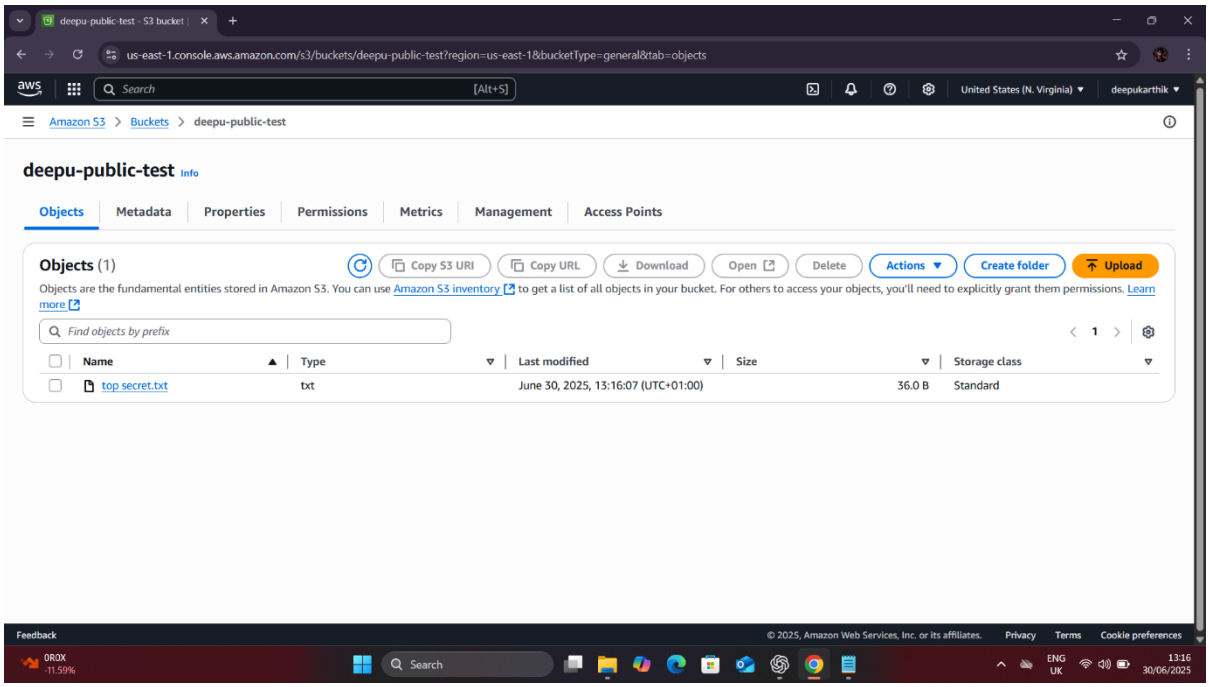


Figure 4.5: Misconfigured bucket listing top secret.txt as publicly accessible

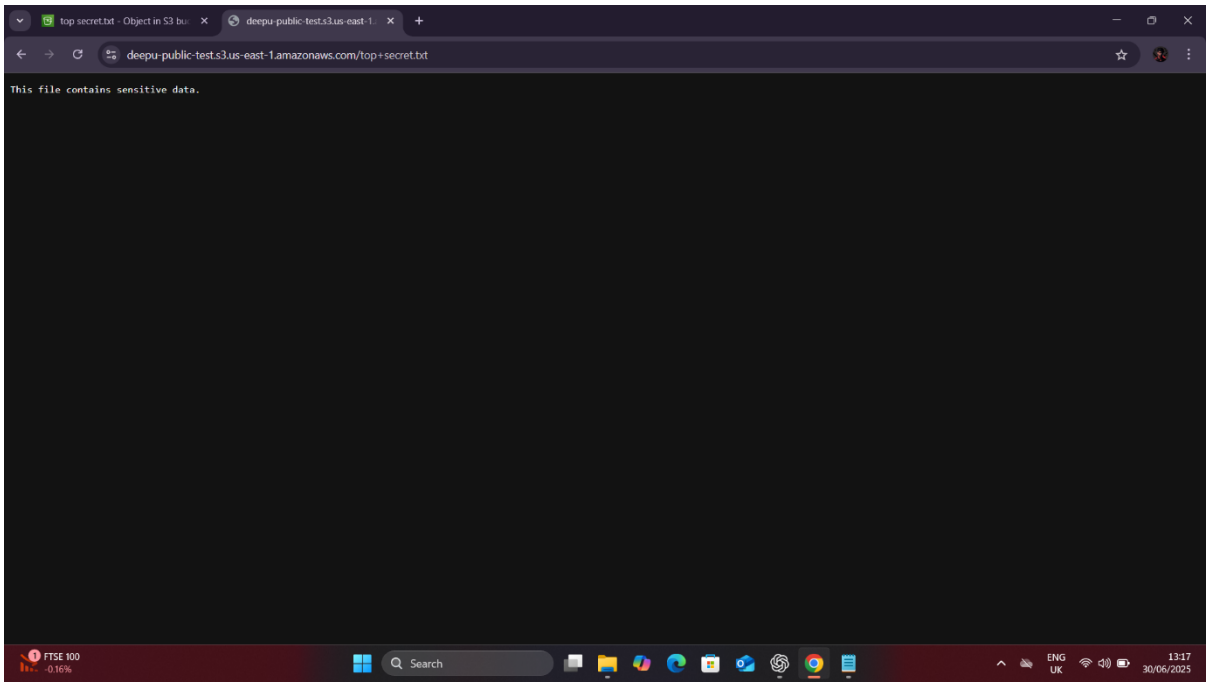


Figure 4.6: Public browser access to sensitive file stored in S3 bucket

The testing host was a Kali Linux virtual machine. Kali was chosen because it consolidates common penetration testing utilities into a single distribution, making it well suited for controlled experimental research. Tools employed included:

- curl for issuing HTTP requests, particularly when querying the EC2 Instance Metadata Service (IMDS) and retrieving publicly available files.
- SSH clients for securely connecting to EC2 instances, installing software such as Apache2, and placing test files within the web root.
- Standard browser access to validate misconfigurations from the perspective of an unauthenticated external user.

The design stage deliberately emphasised simplicity and transparency. By limiting the scope to a small number of services, the experiment was able to isolate security weaknesses that directly map to categories in the OWASP Top Ten (OWASP, 2021). For instance, the S3 configuration was set up to demonstrate Broken Access Control (A01), while the EC2 servers provided opportunities to showcase Security Misconfiguration (A05) and Identification and Authentication Failures (A07).

Overall, the design of the AWS environment balanced realism with academic control. It reflected the configurations most frequently mishandled in industry settings, while remaining small enough to be reproducible by other researchers within the constraints of the AWS Free Tier. The overall design of the experimental environment is illustrated in Figure 4.1. The diagram highlights the relationship between the testing host, the misconfigured S3 bucket, the

EC2 instance running Apache2, its connection to the Instance Metadata Service (IMDSv1), and the attached IAM role with excessive privileges.

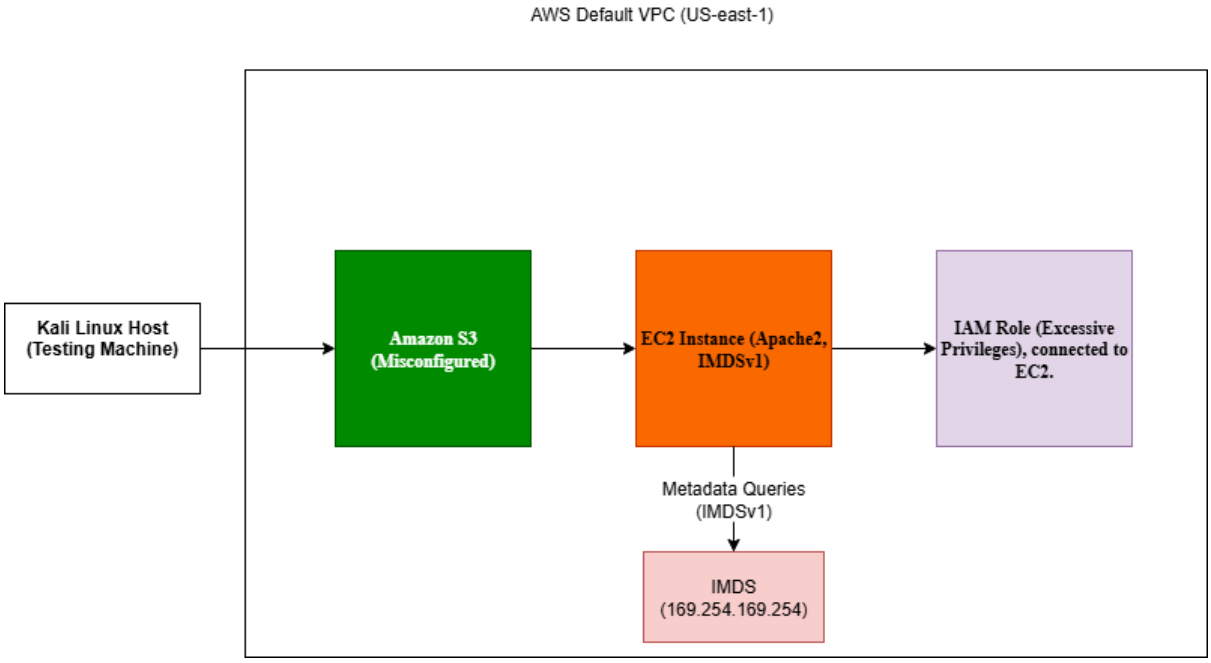


Figure 4.7: AWS lab architecture illustrating intentional misconfigurations and attack paths (Author's own, 2025).

4.4 Misconfigurations Introduced

The misconfigurations introduced in this environment were deliberately selected to replicate vulnerabilities frequently exploited in real world cloud breaches and formally recognised in the OWASP Top Ten (OWASP Foundation, 2021). Each misconfiguration demonstrates a breakdown in fundamental security principles, particularly access control, authentication, and secure configuration management.

To ensure clarity and reproducibility, Table 4.1 summarises the key misconfigurations, their OWASP mappings, and examples of relevant industry cases. The following subsections expand on each misconfiguration in detail, supported by screenshots captured during the implementation process.

Service	Misconfiguration	OWASP Mapping	Real World Case
Amazon S3	Block Public Access disabled; bucket allowed unauthenticated listing and uploads	A01: Broken Access Control, A05: Security Misconfiguration	Accenture (2017) exposed internal credentials
Amazon EC2 (Apache2)	Sensitive file stored in default web root, retrievable over HTTP	A05: Security Misconfiguration, A02: Cryptographic Failures (plaintext secrets)	Multiple cases of exposed config/backup files
Amazon EC2 Metadata Service	IMDSv1 enabled (unauthenticated HTTP access to 169.254.169.254)	A05: Security Misconfiguration, A07: Identification and Authentication Failures	Capital One (2019) – metadata exploited for credentials
IAM Role Permissions	Overly broad permissions (AmazonS3ReadOnlyAccess attached at instance level)	A01: Broken Access Control, A05: Security Misconfiguration	Common in cloud breaches, noted in Palo Alto Unit 42 reports

Table 4.1: Summary of Misconfigurations Introduced in AWS Lab Environment

S3 Public Listing and Write Access

Amazon S3 buckets were configured with Block Public Access disabled, accompanied by policies that allowed unauthenticated users to both list and upload objects. This created a scenario in which sensitive files could be retrieved directly from the internet or malicious files could be uploaded without restriction As shown in Figures 4.2 to 4.5, the bucket accepted file uploads and exposed stored objects publicly, confirming the misconfiguration. Such a configuration exemplifies Broken Access Control (A01), as it enables unauthorised data access, and Security Misconfiguration (A05), since AWS defaults are explicitly designed to prevent this behaviour. Similar lapses have resulted in high profile breaches, such as Accenture's 2017 incident where unsecured S3 buckets exposed internal credentials (UpGuard, 2017).

EC2 Apache2 File Exposure

An EC2 instance running Apache2 was misconfigured to serve files directly from the default web root without additional controls. A file containing simulated credentials was placed in /var/www/html and made available over HTTP. This weakness falls under Security Misconfiguration (A05), reflecting the risks of relying on insecure defaults, and may also overlap with Cryptographic Failures (A02) where plaintext secrets are exposed. Similar exposures have been documented where web servers leaked configuration files or backups, providing attackers with direct pathways to escalate access.

EC2 Metadata Exposure (IMDSv1 Enabled)
The EC2 Instance Metadata Service (IMDS) was left in its legacy IMDSv1 configuration, which allows unauthenticated HTTP requests to the 169.254.169.254 endpoint. This directly maps to Identification and Authentication Failures (A07), as it permits access to sensitive information without verification, and Security Misconfiguration (A05) for failing to enforce IMDSv2. This vulnerability was at the centre of the Capital One breach in 2019, where an attacker retrieved temporary credentials by exploiting unrestricted metadata access (Krebs, 2019).

IAM Role with Excessive Privileges
An IAM role with the managed policy AmazonS3ReadOnlyAccess was attached to an EC2 instance. While seemingly limited, this role violated the principle of least privilege by granting unnecessary access to all buckets in the account. When paired with metadata exposure, this configuration enabled the extraction of temporary credentials. This scenario illustrates Broken Access Control (A01), as permissions exceeded what was operationally required, and Security Misconfiguration (A05) for failing to audit IAM policies effectively. Cloud threat intelligence reports repeatedly emphasise that over privileged IAM roles remain among the most common root causes of cloud compromise (Palo Alto Networks, 2020).

4.5 Ethical and Security Considerations

Conducting penetration testing within a live cloud environment raises ethical concerns, particularly regarding unintended data exposure, compliance with provider policies, and potential collateral impact. To mitigate these risks, the environment was designed with several safeguards.

First, all resources were created within a personal AWS Free Tier account, ensuring that no organisational or production data was involved. Only dummy files were used to simulate sensitive information, such as placeholder credentials or sample “confidential” text. This eliminated the risk of breaching data protection regulations such as GDPR, which impose strict penalties for unauthorised exposure of personal information.

Second, the environment was isolated within the AWS default VPC, meaning all activity was contained within a private network context. No external systems were probed, and no third-party resources were engaged, thereby adhering to the ethical requirement that penetration testing should only be performed on systems owned or explicitly authorised by the tester.

Finally, all testing activities followed responsible disclosure principles. Although no vulnerabilities were reported externally since the environment was intentionally insecure and privately managed the same methodology ensured that any identified issues could be responsibly remediated. The ethical approach taken in this project ensured compliance with both academic research standards and the AWS Acceptable Use Policy.

4.6 Mitigation Frameworks

Although the purpose of this study was to demonstrate how misconfigurations can be exploited, it is equally important to outline mitigation strategies. The vulnerabilities

introduced in this environment correspond to well documented weaknesses, for which effective countermeasures are already established in both AWS security documentation and wider industry frameworks.

For S3 misconfigurations, the primary safeguard is enforcing Block Public Access at the account and bucket level. In addition, IAM policies should follow the principle of least privilege, ensuring that only explicitly authorised users or roles can access objects. AWS Config and Security Hub can automate detection and remediation by flagging publicly accessible buckets in real time.

For EC2 Apache2 misconfigurations, administrators should avoid storing sensitive files within the web root and implement strict file permissions to limit exposure. Web server hardening guides recommend restricting directory access, disabling directory listing, and using network firewalls to restrict HTTP access to trusted IP ranges.

For EC2 metadata exposures, AWS explicitly recommends enabling IMDSv2, which requires the use of session based tokens to access metadata. Organisations should enforce IMDSv2 across all instances via service control policies and audit compliance regularly. Local firewalls or host based intrusion prevention systems can provide an additional safeguard by blocking metadata endpoint queries from untrusted sources.

For IAM role misconfigurations, the principle of least privilege must be rigorously applied. Permissions should be scoped narrowly, temporary credentials should be monitored, and IAM policies should undergo continuous auditing using tools such as IAM Access Analyzer. Excessive permissions represent one of the most significant avenues for privilege escalation and must be minimised wherever possible.

Mapping these mitigations back to OWASP demonstrates that although cloud misconfigurations manifest in unique ways, the underlying remedies remain consistent with general secure design principles. Whether the issue is Broken Access Control, Security Misconfiguration, or Identification and Authentication Failures, the central lesson is that enforcing least privilege, validating configuration, and adopting defence in depth strategies can substantially reduce risk.

4.7 Chapter Summary

This chapter described the design and implementation of a deliberately misconfigured AWS environment, focusing on Amazon S3 buckets and EC2 instances. Four categories of misconfiguration were introduced: public access to S3 buckets, exposure of sensitive files in Apache2, unrestricted metadata services, and over-privileged IAM roles. Each was carefully aligned with real world cloud breaches and mapped to categories in the OWASP Top Ten, demonstrating that cloud specific vulnerabilities are extensions of broader systemic weaknesses such as Broken Access Control (A01) and Security Misconfiguration (A05).

Ethical safeguards were embedded throughout the implementation, ensuring that the environment remained isolated, reproducible, and compliant with academic research

standards. Mitigation strategies were also considered, drawing on AWS documentation, CIS benchmarks, and industry reports to outline how organisations can avoid similar misconfigurations in practice.

By the end of this implementation, a controlled environment had been established that mirrored the conditions observed in cloud breaches such as Capital One (2019) and Accenture (2017).

Chapter 5: Testing and Results

5.1 Introduction

This chapter presents the penetration testing conducted against the intentionally misconfigured AWS environment implemented in Chapter 4. The objective of these tests was to simulate how common cloud misconfigurations in Amazon S3 buckets and EC2 instances can be exploited to compromise confidentiality, integrity, and availability of cloud resources.

Each test was structured to demonstrate the consequences of a specific misconfiguration, evaluate its alignment with recognised vulnerability categories such as the OWASP Top Ten (2021), and identify corresponding mitigation strategies. The findings were then analysed in comparison with real world breaches, such as the Capital One incident in 2019 and the Accenture S3 exposure in 2017, to contextualise the risks within industry practice.

The chapter is divided into two main parts. Section 5.2 outlines the testing methodology, including the penetration testing approach, tools, and ethical considerations. Sections 5.3 to 5.7 present the results of five tests targeting S3 and EC2 vulnerabilities, while Section 5.8 synthesises the findings to identify patterns and their relationship to industry case studies.

5.2 Testing Methodology

The penetration testing followed a structured methodology informed by the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment (Scarfone et al., 2008) and the OWASP Testing Guide v4 (OWASP, 2020). This ensured the process was systematic, reproducible, and aligned with widely recognised best practices.

5.2.1 Testing Approach

Three common penetration testing approaches are typically recognised:

- Black-box testing: where the tester has no prior knowledge of the system, simulating an external attacker.
- White-box testing: where the tester has full knowledge of the system, simulating an insider or developer.
- Grey-box (or red-team) testing: where the tester has partial knowledge, reflecting the perspective of an attacker with limited insider information.

This study primarily adopted a white-box approach, as the misconfigured environment was deliberately created and controlled by the researcher. Full knowledge of its architecture and settings ensured each vulnerability could be replicated transparently. However, individual test procedures were executed in a black-box manner where appropriate for example, retrieving

objects from S3 or files from Apache2 without authentication to simulate how an external attacker would exploit the weaknesses.

5.2.2 Testing Phases

In line with NIST SP 800-115, each test was conducted in four phases:

1. Planning and Preparation – Defining scope and ensuring ethical boundaries.
2. Discovery – Identifying exploitable misconfigurations.
3. Attack/Exploitation – Attempting to exploit weaknesses using penetration testing tools.
4. Reporting and Documentation – Recording outcomes, analysing risks, and mapping to OWASP Top Ten categories.

5.2.3 Tools and Environment

Testing was performed from a Kali Linux virtual machine, chosen for its integration of security tools. The key utilities included:

- curl for HTTP requests, particularly to the EC2 Instance Metadata Service.
- AWS CLI for interacting with misconfigured buckets and IAM roles.
- SSH clients for connecting to EC2 instances to configure and monitor services.
- Web browsers for testing unauthenticated access to S3 objects and Apache2 web content.

5.2.4 Ethical Considerations

All testing was confined to a private AWS Free Tier account within the default VPC in the us-east-1 region. Only dummy files containing placeholder credentials or generic text were used to simulate sensitive data, ensuring no personal or production information was at risk. In addition, the testing adhered strictly to AWS's Acceptable Use Policy, with no interaction beyond the researcher's controlled environment.

5.3 Test 1: S3 Public Listing and Read Access

Objective

This test examined the risk associated with disabling Block Public Access on an Amazon S3 bucket, thereby permitting public object listing and unauthenticated read access. Such misconfigurations have historically led to data leaks, with attackers able to enumerate and retrieve sensitive information without restriction.

Methodology

An S3 bucket named deepu-public-test was created with Block Public Access disabled. This configuration allowed any external user to view and retrieve objects without authentication.

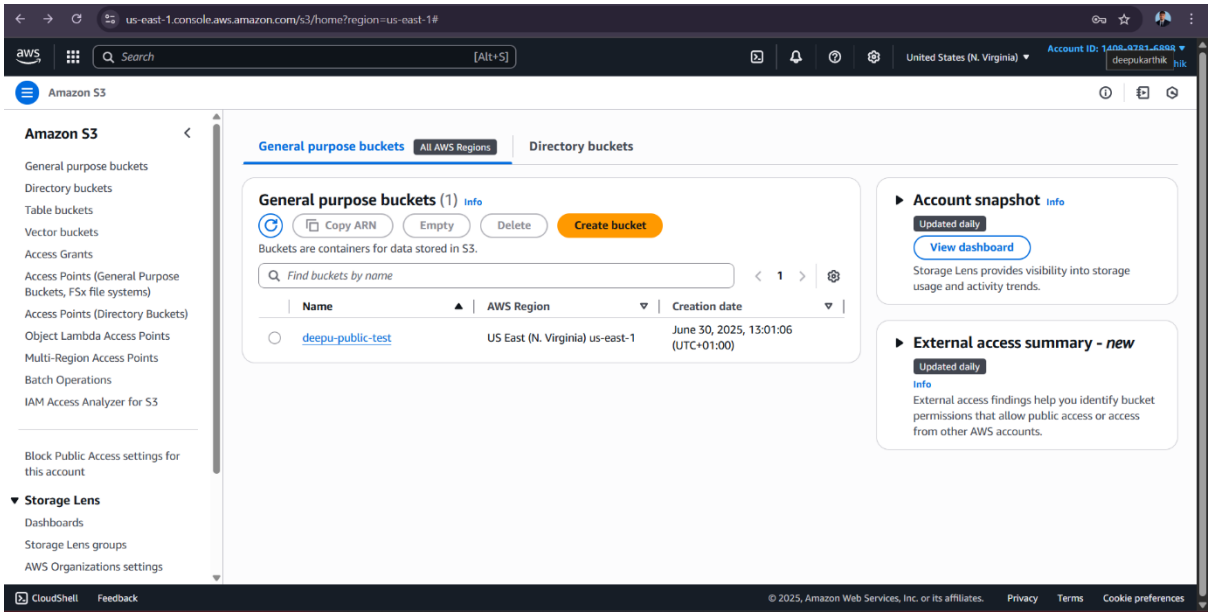


Figure 5.1 illustrates the bucket creation process with public access controls explicitly turned off.

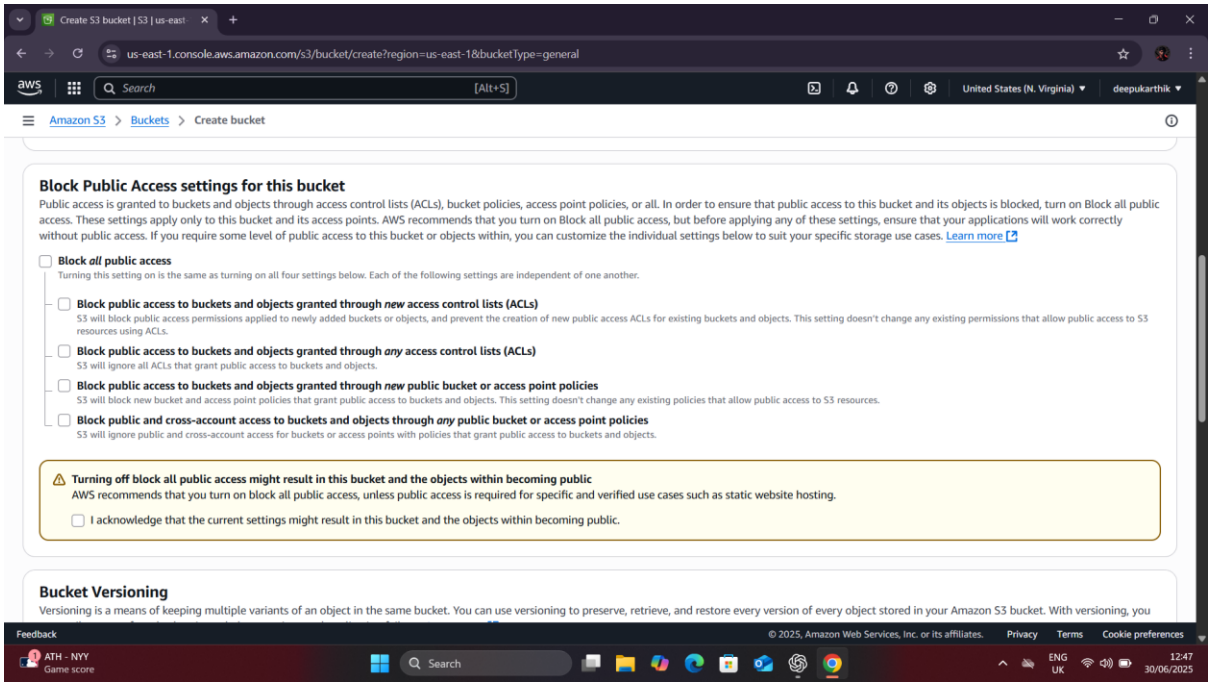


Figure 5.1: S3 bucket created with Block Public Access disabled (Author's own, 2025).

To simulate sensitive content, a text file named top secret.txt was uploaded into the bucket. This upload is shown in Figure 5.2.

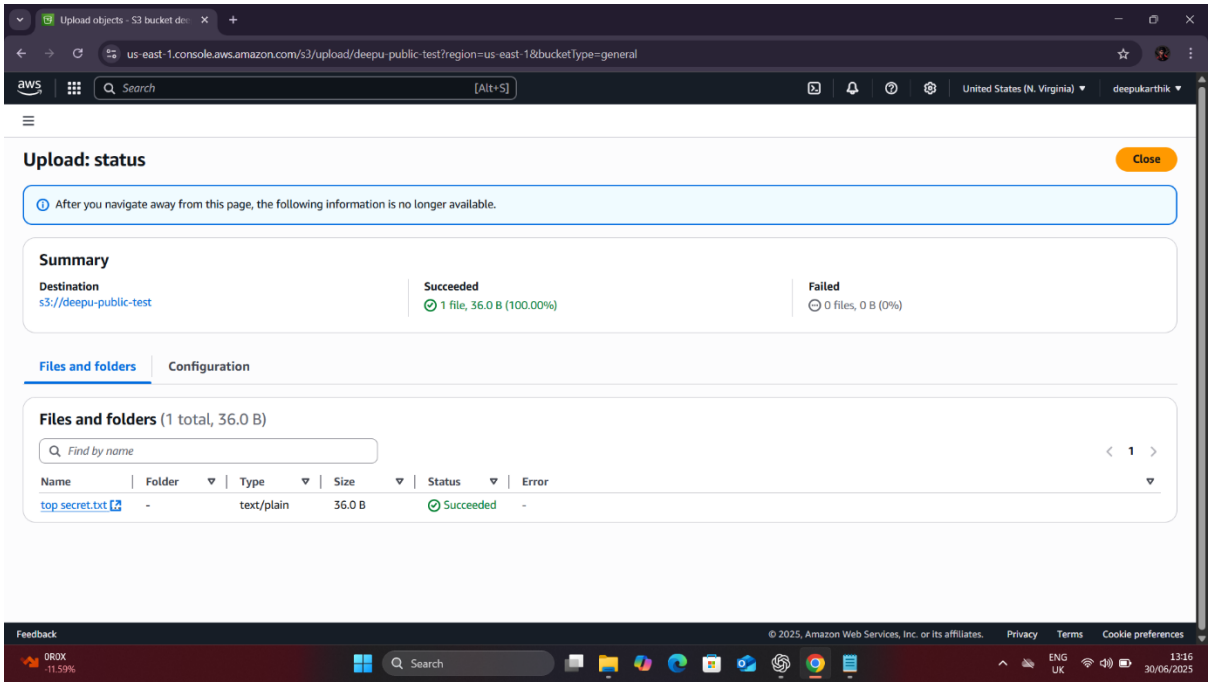


Figure 5.2: Dummy sensitive file uploaded to misconfigured S3 bucket (Author's own, 2025).

Testing was then conducted in a black-box manner to replicate how an external attacker would interact with the bucket. The file's object URL was copied from the AWS console and accessed through:

1. A standard web browser in an unauthenticated session.
2. The curl command in Kali Linux to demonstrate automated retrieval.

Results

The file was successfully retrieved in both scenarios. Figure 5.3 shows the unauthenticated browser session accessing the file directly from its public URL. Figure 5.4 demonstrates retrieval of the same file via curl in Kali Linux.

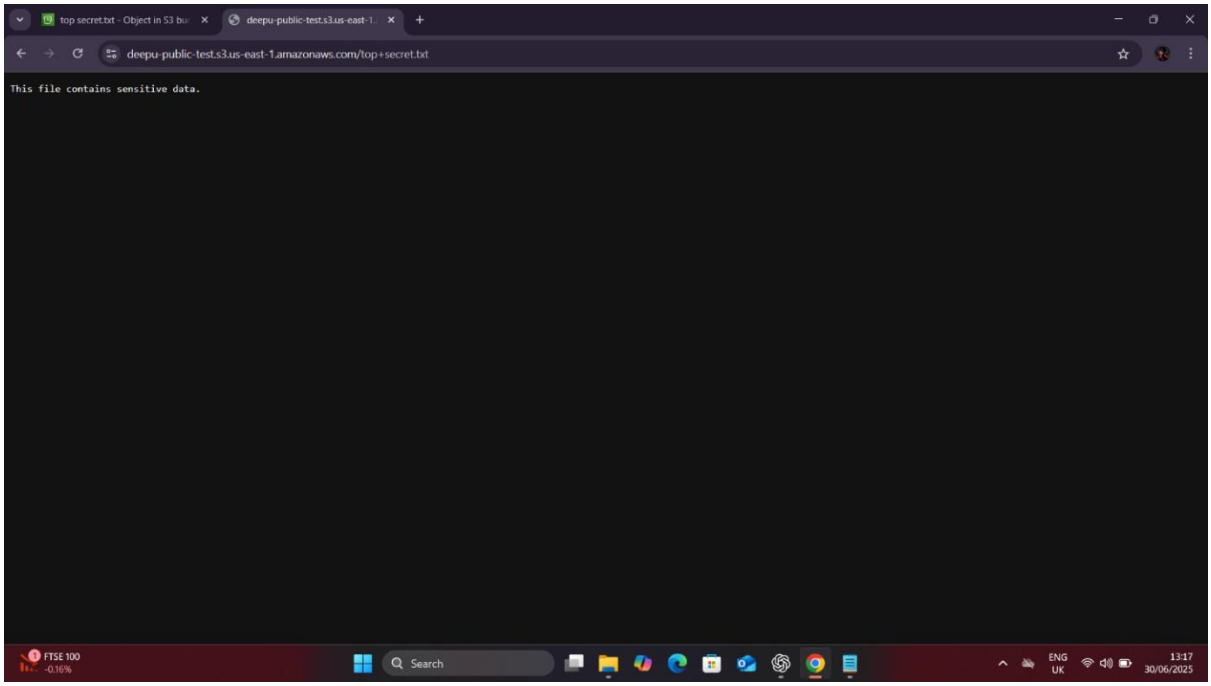


Figure 5.3: Public retrieval of file via browser without authentication.

Analysis

The results confirm that disabling Block Public Access leads to unauthorised data exposure. This misconfiguration directly violates the principle of least privilege and maps to two OWASP Top Ten categories: A01: Broken Access Control and A05: Security Misconfiguration (OWASP, 2021). In a real world context, such vulnerabilities have been exploited in breaches including Accenture's 2017 exposure, where unsecured S3 buckets revealed API keys and sensitive internal data (UpGuard, 2017).

Mitigation

To prevent this vulnerability, organisations should enforce Block Public Access at both the bucket and account level, ensuring it cannot be accidentally disabled. IAM policies must be scoped according to the principle of least privilege, and continuous monitoring should be implemented via AWS Config rules or AWS Security Hub to detect and remediate misconfigured buckets automatically.

5.4 Test 2 : S3 Bucket Misconfiguration Testing

Amazon S3 is frequently identified as one of the most misconfigured AWS services due to its flexible access policies and integration with public cloud applications. In this test, a bucket was deliberately provisioned with public write permissions to simulate a real world scenario in which attackers can tamper with or inject files into an organisation's cloud storage.

Exploitation Process

After disabling the Block Public Access settings during bucket creation, a file named Attacker-file.txt was uploaded to the bucket using the AWS Management Console (Figure 5.1). The file contained the simple payload message: “This was uploaded by an attacker.” This step simulated unauthorised modification of stored data.

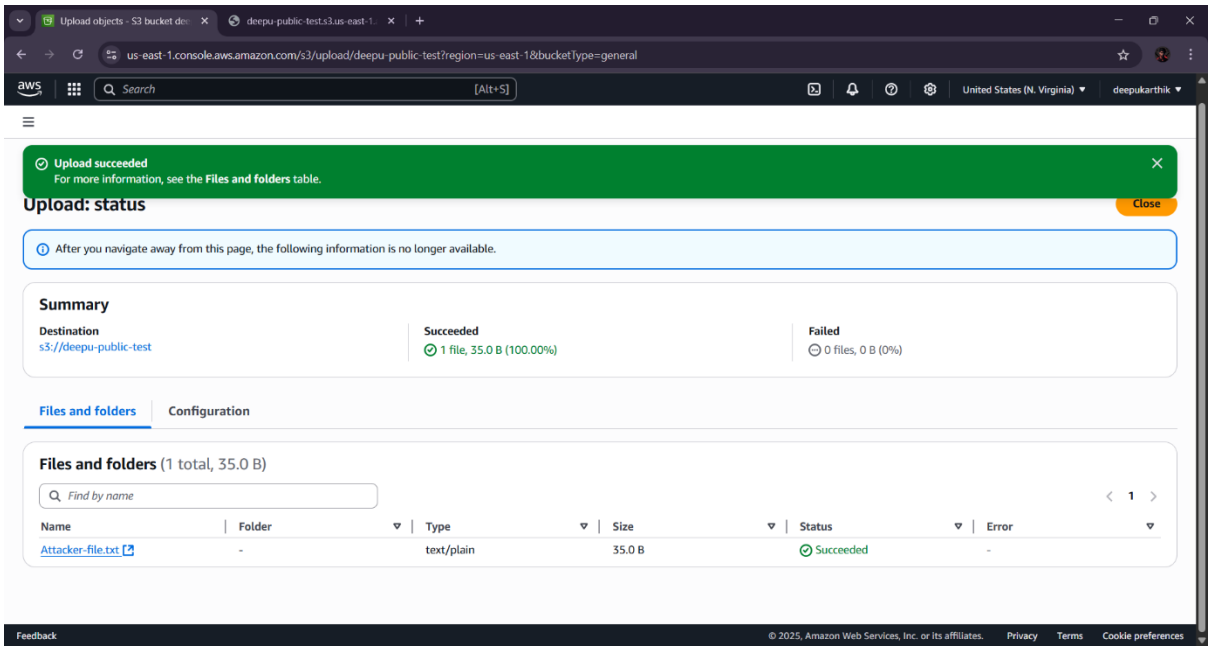


Figure 5.1: Successful upload of malicious file to misconfigured bucket.

The successful upload was confirmed in the AWS console, where the object appeared under the bucket’s object list with a “Succeeded” status.

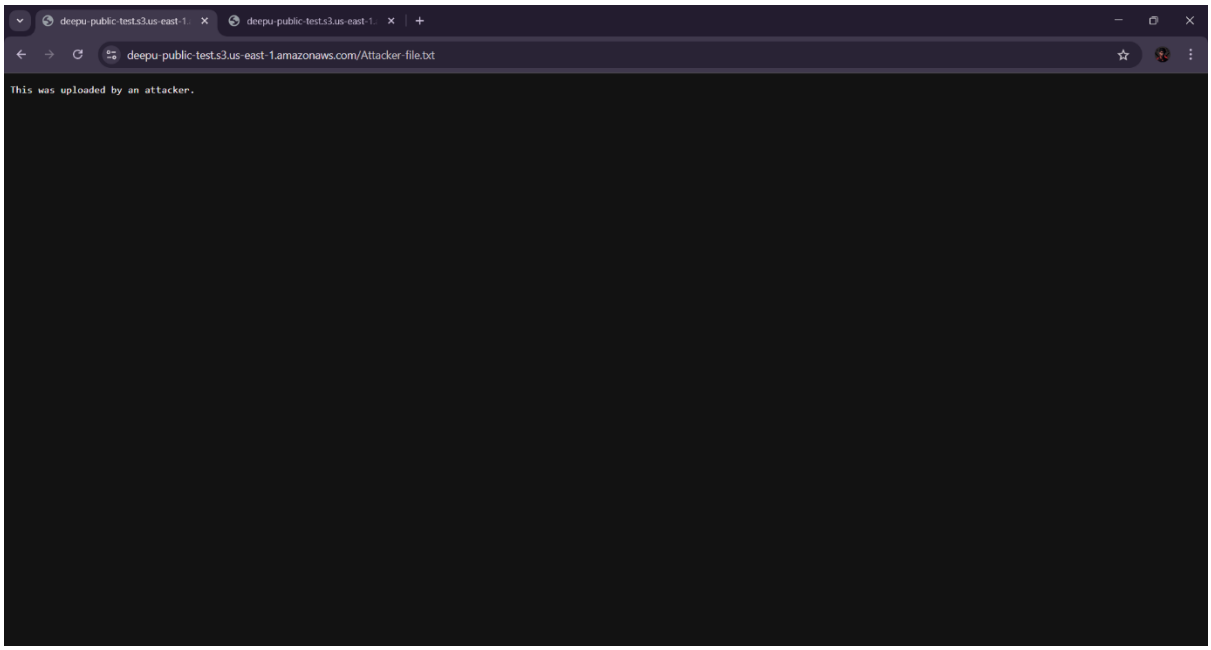


Figure 5.2 Public retrieval of attacker file via browser

Subsequently, the file was retrieved directly from the bucket's public URL using both command-line and browser-based methods. The curl command executed from the Kali Linux host successfully displayed the injected text file, verifying public accessibility of attacker-controlled content.

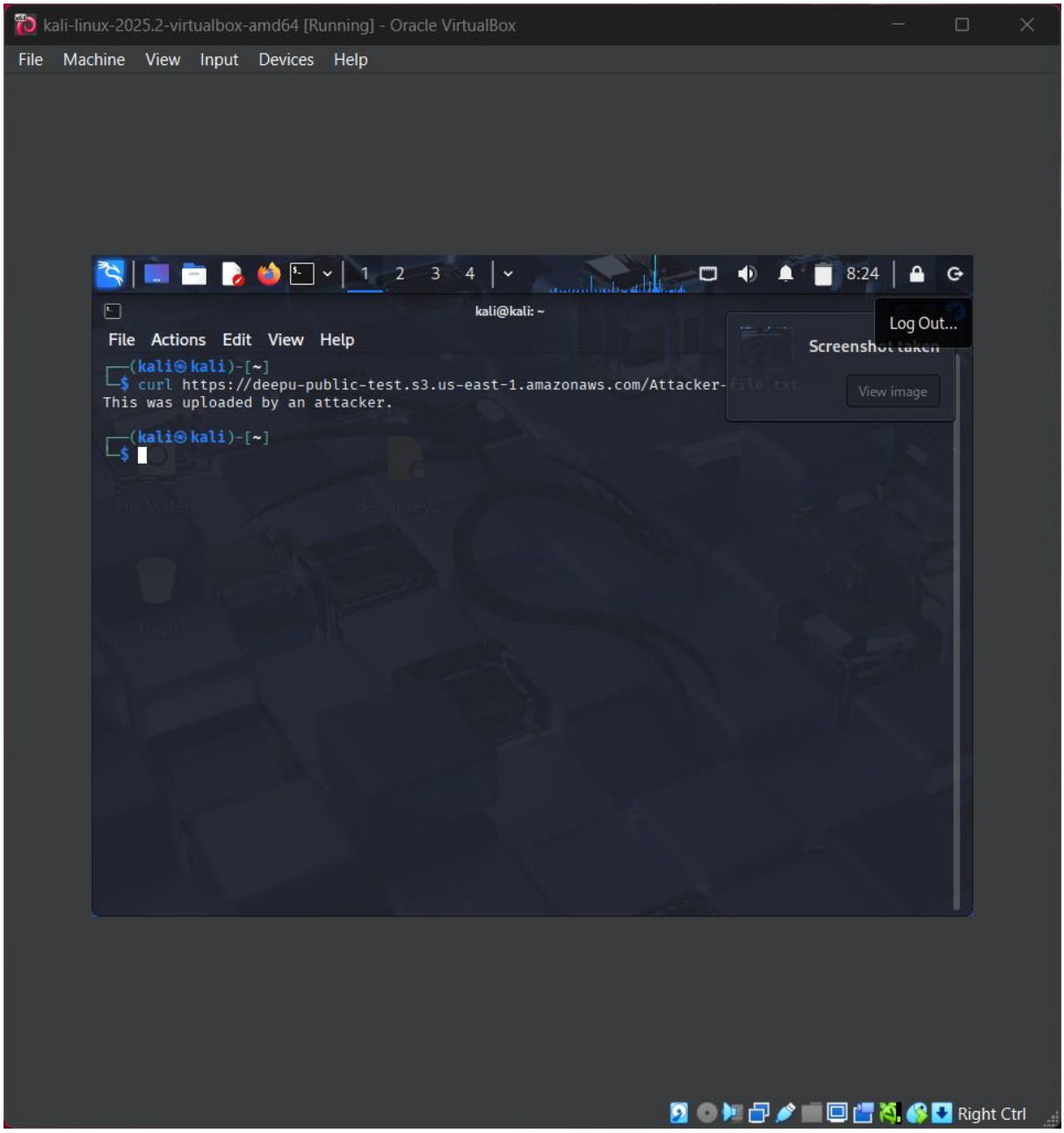


Figure 5.3: Public retrieval of attacker file via curl from Kali Linux

The same file was accessed through a web browser by navigating to the S3 object's public link, which again revealed the attacker's payload (Figure 5.4).

Analysis

This test confirmed the risk posed by unrestricted public access in S3. If exploited in a real-world environment, attackers could inject malware, replace critical files, or host malicious content using an organisation's trusted cloud domain. Such weaknesses fall under OWASP

Top 10 A05: Security Misconfiguration (OWASP, 2021), where misapplied permissions expose cloud assets to unauthorised parties.

The scenario closely mirrors real world incidents, such as the 2017 Accenture breach, in which UpGuard researchers discovered publicly accessible S3 buckets containing internal credentials (UpGuard, 2017). This highlights that even global enterprises are susceptible to misconfigured S3 storage.

Mitigation

To mitigate this risk, organisations should first enable Block Public Access at both the account and bucket level, which prevents unauthorised users from writing to S3 buckets. Access permissions should be tightened through the principle of least privilege, ensuring that only explicitly authorised IAM principals are able to upload files. In addition, automated safeguards such as AWS Config rules and Security Hub can be used to detect and alert on publicly writable buckets as soon as they appear. Finally, continuous monitoring should be enforced by enabling logging through AWS CloudTrail and S3 server access logs, providing visibility into all access attempts. When applied together, these measures effectively close the attack vector demonstrated in this test and substantially reduce the likelihood of unauthorised data manipulation.

5.5 Test 3 : EC2 Apache2 Sensitive File Exposure

Objective

The objective of this test was to determine whether misconfigured Apache2 web server settings on an EC2 instance could expose sensitive files to unauthenticated users. This vulnerability relates directly to OWASP Top 10: A5 Security Misconfiguration (OWASP, 2021), since insecure defaults and lack of proper file restrictions allow attackers to access sensitive data.

Implementation

An EC2 instance was launched in the us-east-1 region using the Amazon Machine Image (AMI) ami-053bd05c279acc90 with a t2.micro instance type. The instance was deployed within the default VPC and assigned a security group that allowed inbound HTTP (port 80) traffic from any IP address. Apache2 was installed and configured to serve files from the /var/www/html directory without additional access restrictions. A file named confidential.txt containing simulated sensitive credentials was placed in the web root.

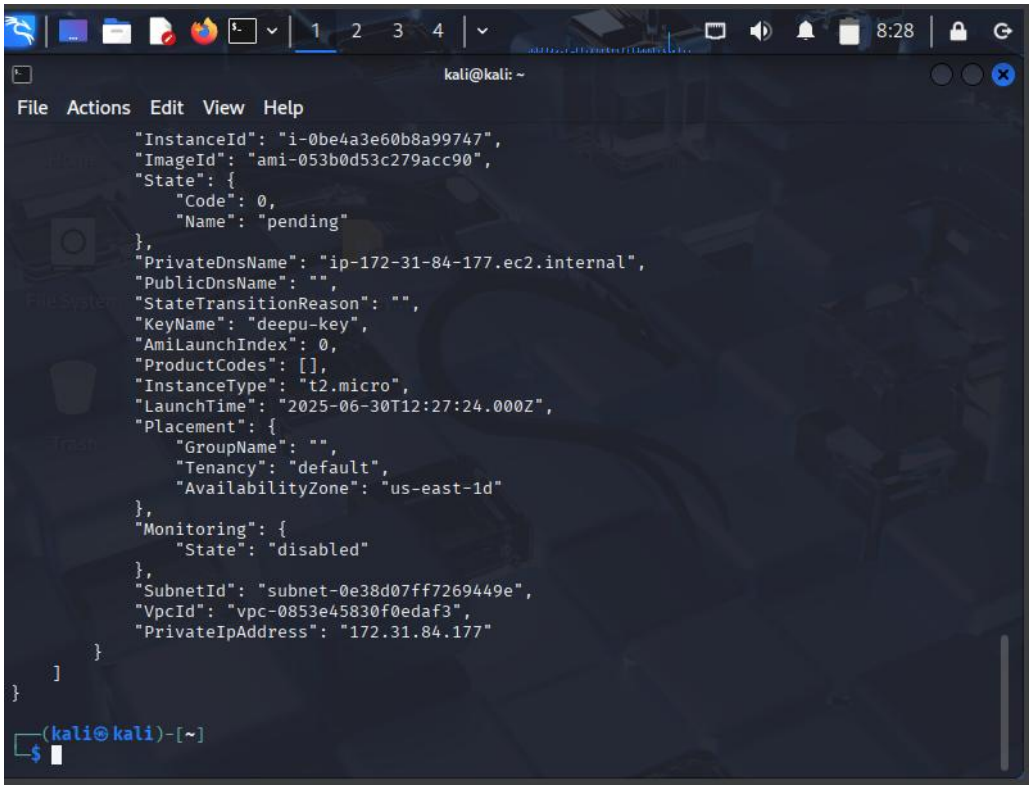


Figure 5.4: EC2 instance metadata showing configuration details

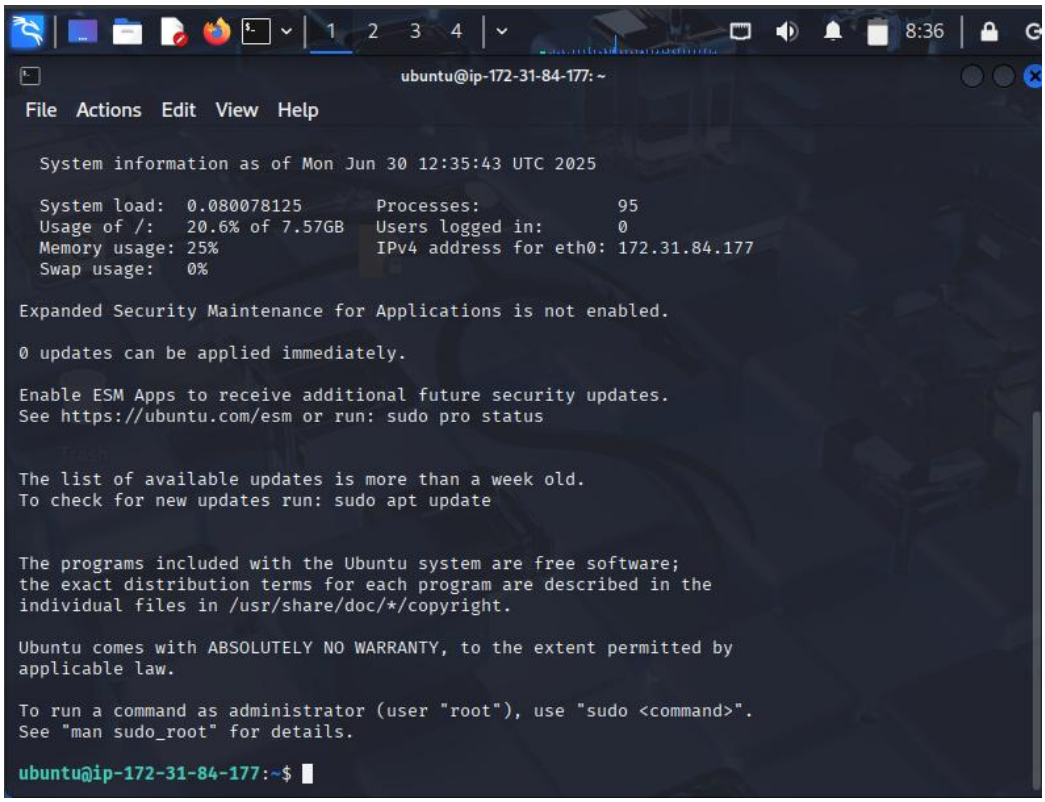


Figure 5.5: SSH connection to EC2 instance

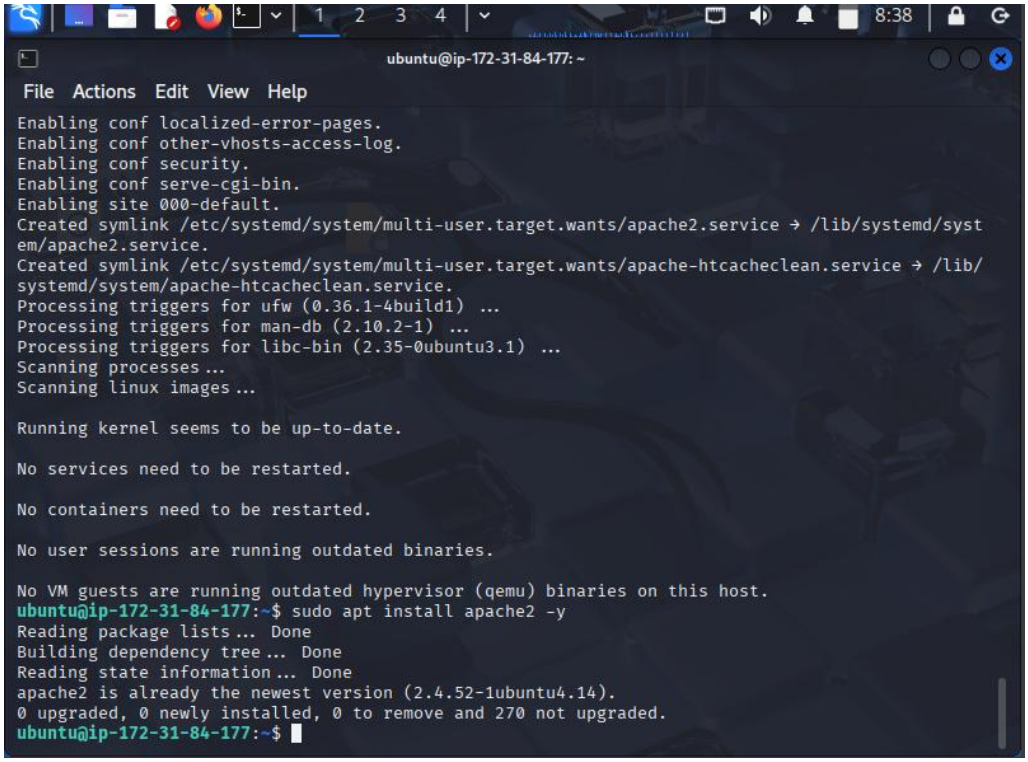


Figure 5.6: Apache2 installation and verification

Testing Procedure

1. Launched the EC2 instance and connected via SSH from the Kali Linux environment.
2. Installed Apache2 using sudo apt install apache2 -y and confirmed the service was active.
3. Created a text file confidential.txt in the /var/www/html directory containing the message "This file contains sensitive credentials.".
4. Accessed the file locally on the server via curl http://localhost/confidential.txt to confirm it was served by Apache2.
5. Retrieved the file remotely by navigating to the EC2 instance's public IP address in a web browser.

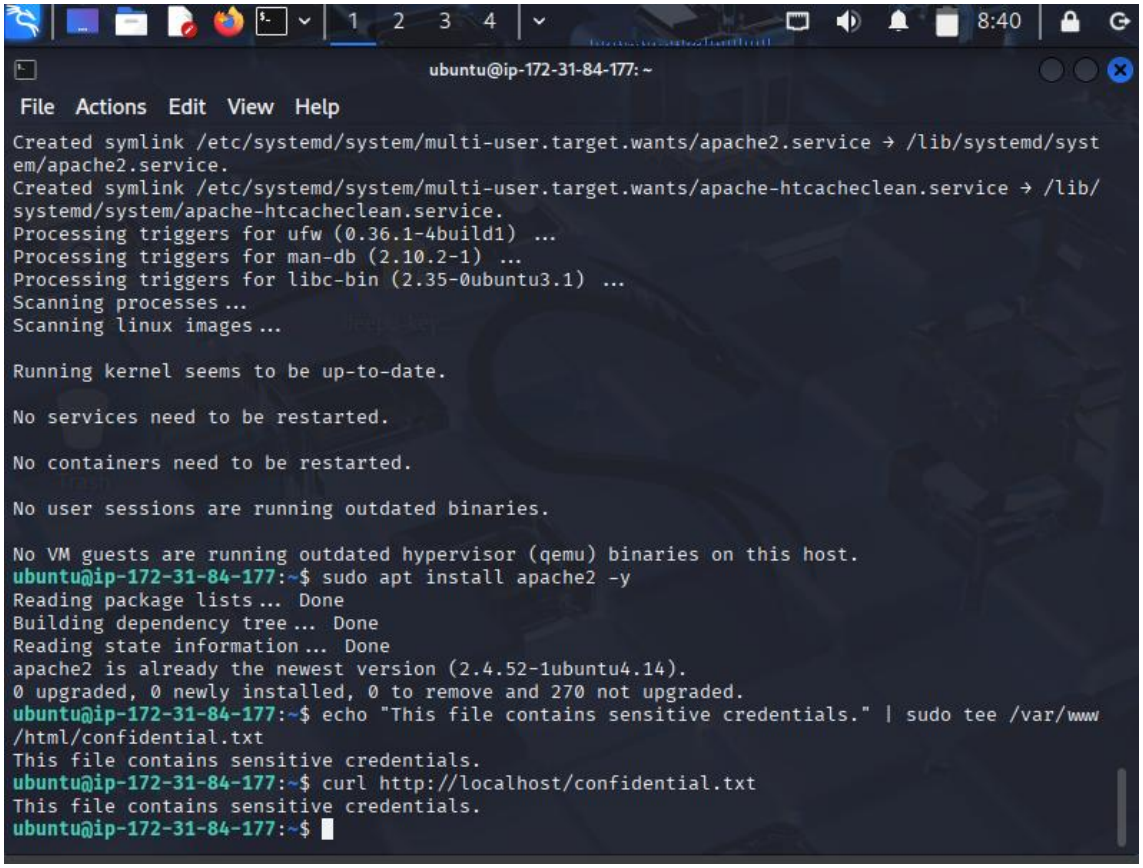


Figure 5.7: Sensitive file created in Apache2 web root and accessed locally

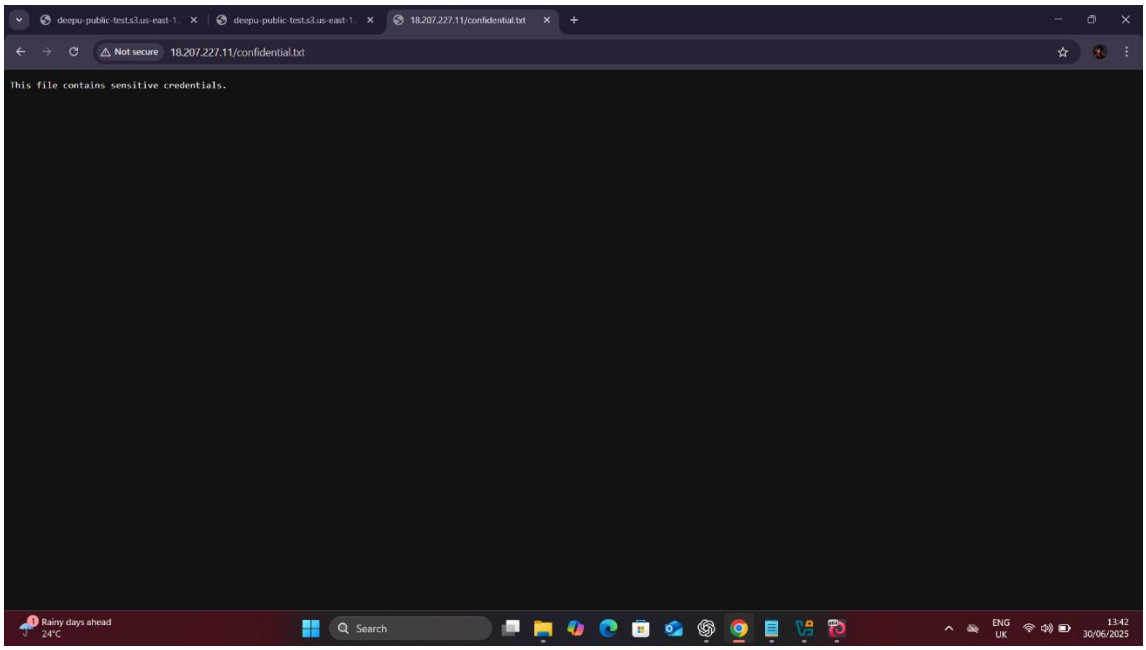


Figure 5.8: Remote access to sensitive file via EC2 public IP

Results

The confidential file was accessible from any internet connected device without authentication. This demonstrated that sensitive files placed in the Apache web root directory were publicly retrievable, replicating a real world data leakage scenario.

Analysis
This misconfiguration could lead to severe confidentiality risks, as attackers might access credentials, configuration files, or backups. Such exposures are common in production environments when administrators fail to restrict file access or mistakenly place sensitive data in public directories.

Mitigation
To prevent such risks, organisations should ensure sensitive files are never stored in web server roots and apply strict file permissions to directories served by Apache. AWS Security Groups should restrict inbound access to only necessary IP ranges and ports, reducing public exposure. Further protection can be achieved by hardening Apache configurations, deploying AWS WAF to filter malicious traffic, and enabling monitoring tools such as AWS CloudTrail and GuardDuty to detect unusual access patterns.

5.6 Test 4 : EC2 Metadata Exposure

Objective

The objective of this test was to assess whether an EC2 instance's metadata service (IMDS) was exposed in a way that allowed unauthenticated access to sensitive instance information. This vulnerability maps to OWASP Top 10: A5 Security Misconfiguration (OWASP, 2021), since unrestricted access to the metadata endpoint can expose credentials, tokens, or configuration details.

Implementation

An EC2 instance was launched in the us-east-1 region using AMI ami-053bd05c279acc90 with instance type t2.micro. The security group was configured to allow inbound SSH (port 22) and HTTP (port 80) from any source. The instance was deployed without enforcing IMDSv2, allowing metadata to be accessed using unauthenticated HTTP requests to the special link local address 169.254.169.254.

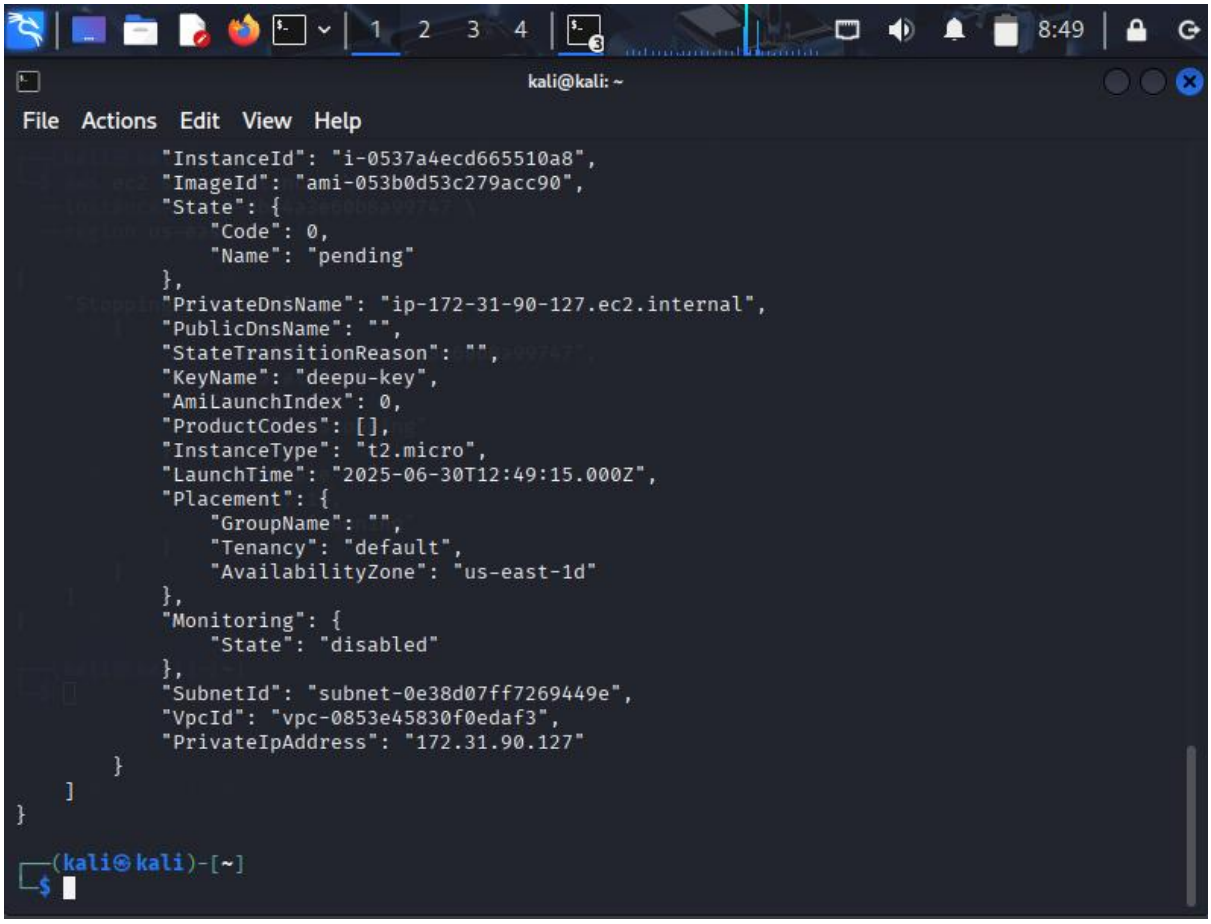


Figure 5.9: EC2 instance metadata and configuration details

Testing Procedure

1. Established an SSH connection to the EC2 instance from a Kali Linux machine (Figure 4.14).
2. Queried the EC2 metadata service using curl http://169.254.169.254/latest/meta-data/ to retrieve available metadata categories (Figure 4.15).
3. Extracted specific metadata entries, including the public IPv4 address, by querying curl http://169.254.169.254/latest/meta-data/public-ipv4 (Figure 4.16).

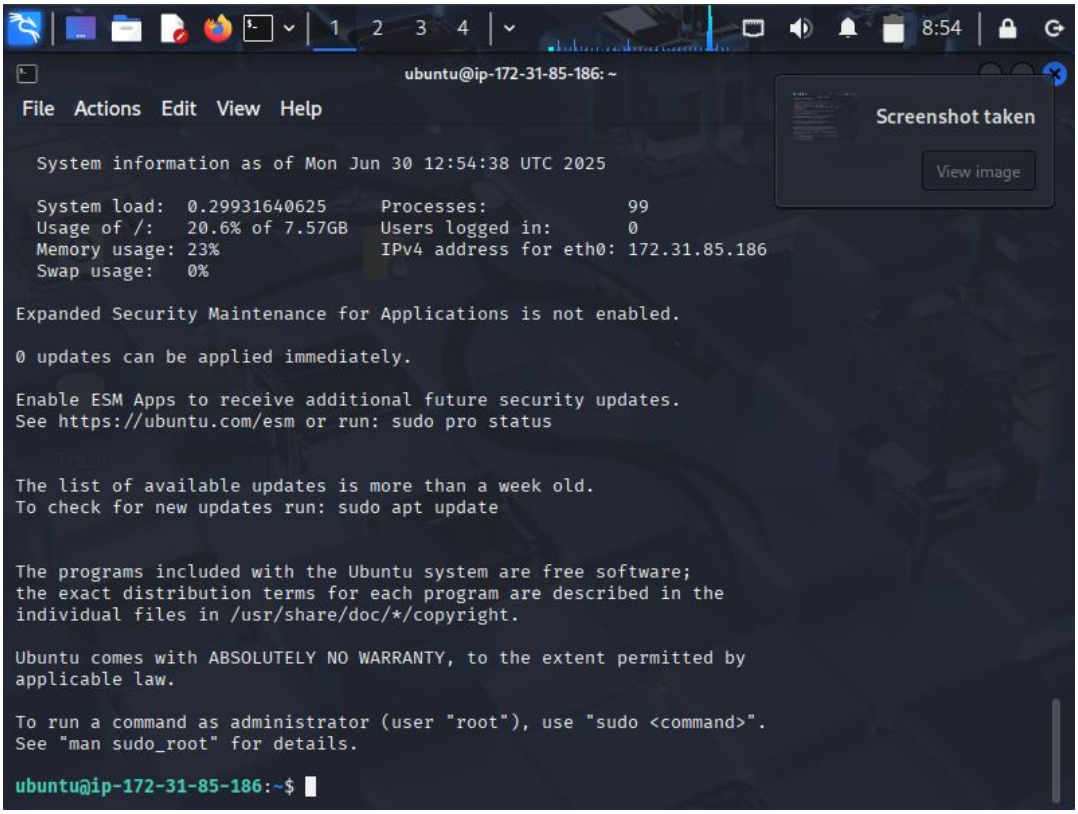


Figure 5.10 : SSH session established to EC2 instance

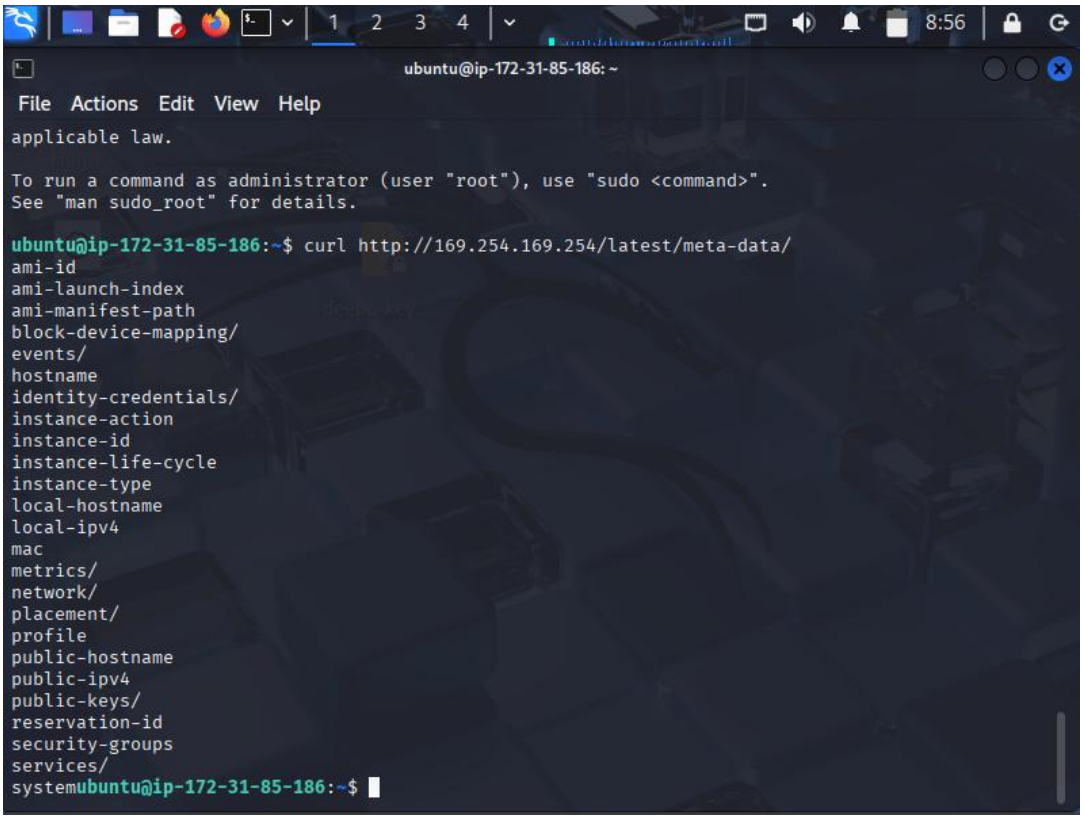


Figure 5.11: Retrieving available metadata categories via curl

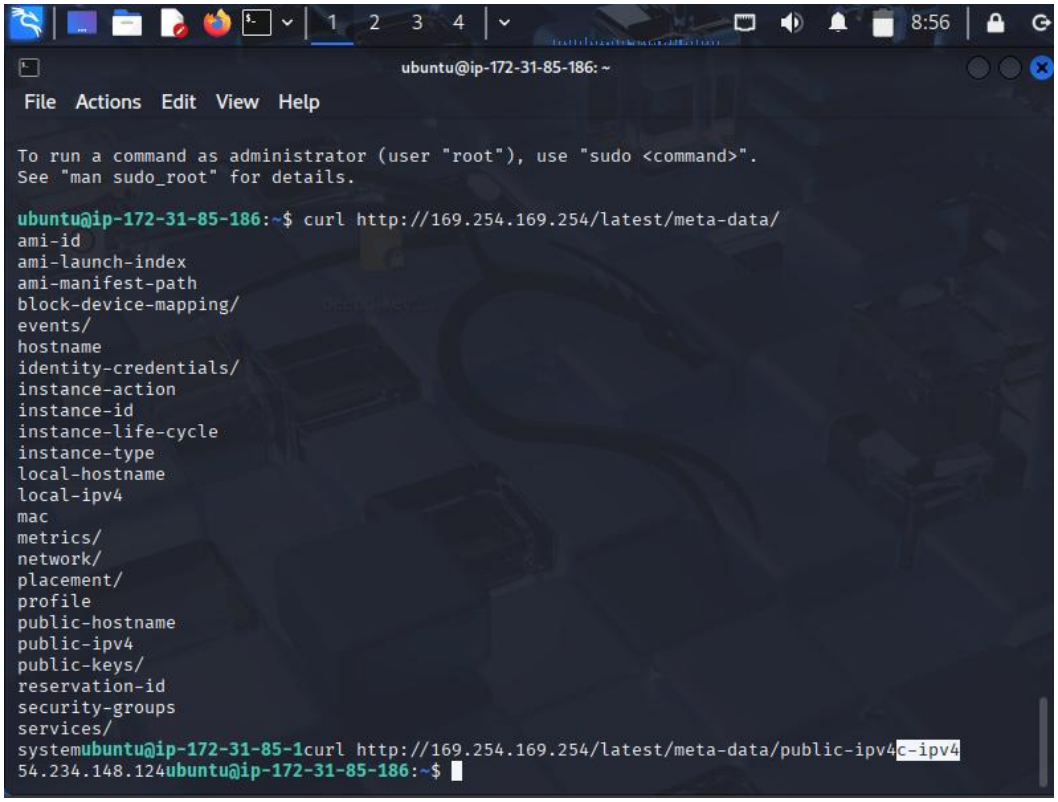


Figure 5.12: Extracting public IPv4 address from metadata service

Results

The metadata service responded without restrictions, providing detailed information about the instance's network, storage, and identity data. This confirmed that the EC2 instance was configured with open metadata access using IMDSv1, which does not enforce session authentication.

Analysis

This misconfiguration poses serious risks. Attackers who gain remote code execution (RCE) on the server could extract IAM role credentials from the metadata endpoint. Such stolen credentials could then be used to access AWS resources, escalate privileges, or exfiltrate data. Real world breaches (e.g., Capital One, 2019) were directly linked to this type of metadata exposure.

Mitigation

The risks associated with unrestricted access to the EC2 instance metadata service can be effectively reduced through a combination of configuration hardening and monitoring. The most important step is enforcing IMDSv2, which requires session based authentication and prevents unauthenticated queries to the metadata endpoint. In addition, access to the 169.254.169.254 address should be restricted using host based firewalls such as iptables, thereby ensuring that only authorised processes within the instance can query the service. The application of the principle of least privilege to IAM roles further limits the damage that

could result from stolen credentials, as attackers would only gain access to narrowly defined resources. Finally, continuous monitoring should be enabled by logging instance activity with AWS CloudTrail and detecting anomalous metadata queries using AWS GuardDuty, allowing organisations to respond rapidly to potential exploitation attempts.

5.7 Test 5 : IAM Role Metadata Exposure

Objective

The objective of this test was to determine whether the EC2 Instance Metadata Service (IMDS) could be exploited to retrieve sensitive information, such as IAM role credentials, without requiring authentication. This vulnerability is directly related to OWASP Top 10: A5 Security Misconfiguration (OWASP, 2021), since unrestricted metadata access allows attackers to escalate privileges and pivot within the AWS environment.

Implementation

A new IAM role named EC2ReadOnlyRole was created and assigned the AWS managed policy AmazonS3ReadOnlyAccess, granting read access to all S3 buckets in the account. This role was then attached to an EC2 instance launched in the us-east-1 region without IMDSv2 enforcement, allowing metadata queries to return security credentials.

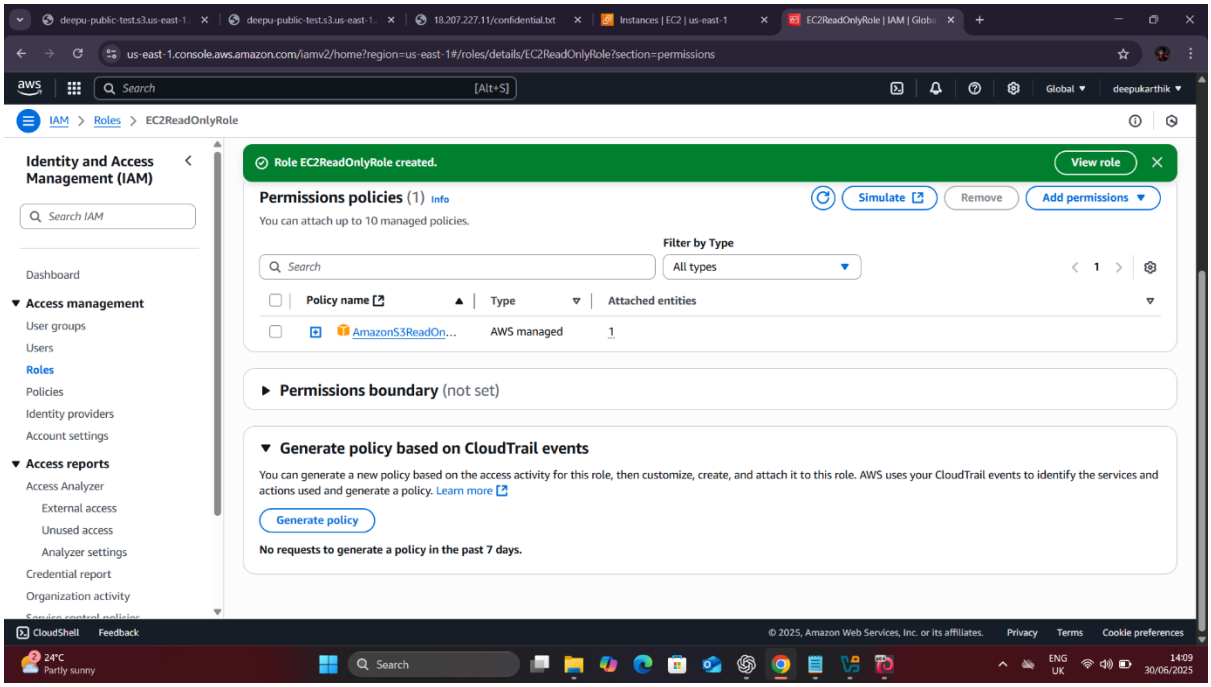


Figure 4.17: IAM role with AmazonS3ReadOnlyAccess policy attached

Testing Procedure

1. Launched the EC2 instance and verified its metadata configuration from the Kali Linux host (Figure 4.18).
2. Connected to the instance via SSH and queried the IAM role name assigned to it using

   curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ (Figure 4.19).
3. Requested the full set of temporary credentials for the role by querying: curl http://169.254.169.254/latest/meta-data/iam/security-credentials/EC2ReadOnlyRole (Figure 4.20).
4. Captured the returned AccessKeyId, SecretAccessKey, and Token values.

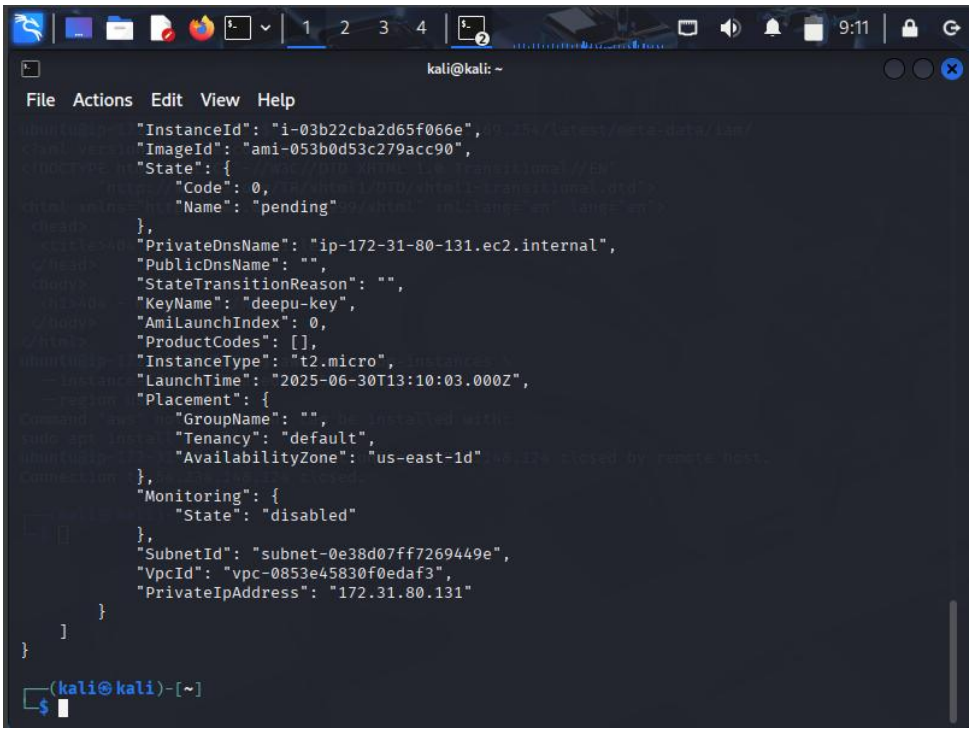


Figure 4.18: EC2 instance details queried from Kali Linux

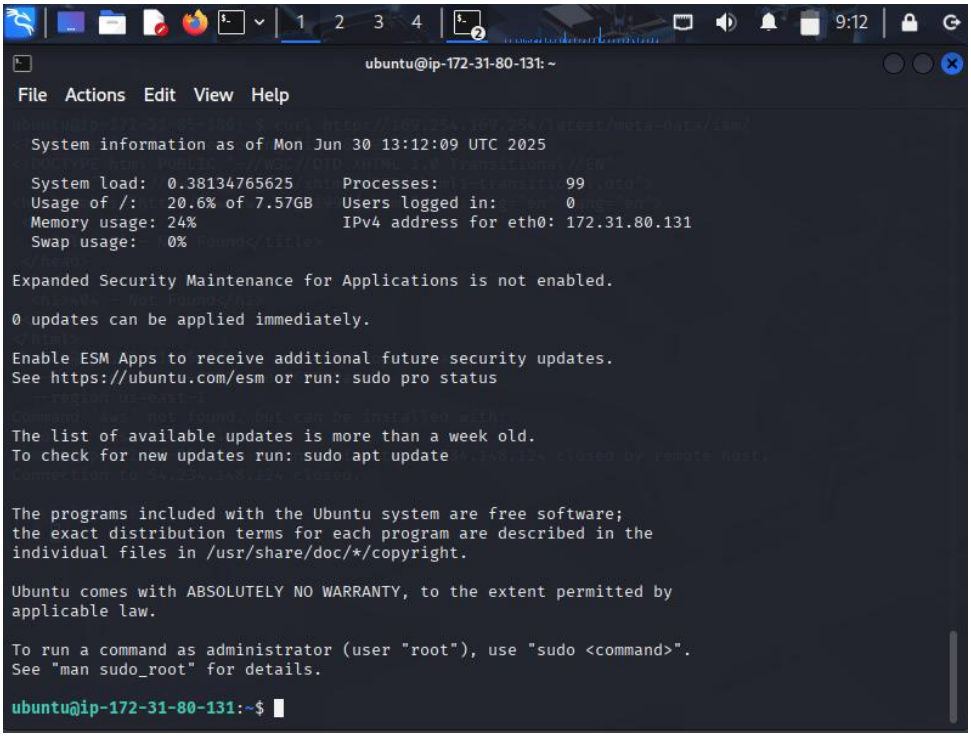


Figure 4.19: Retrieving assigned IAM role name via metadata service

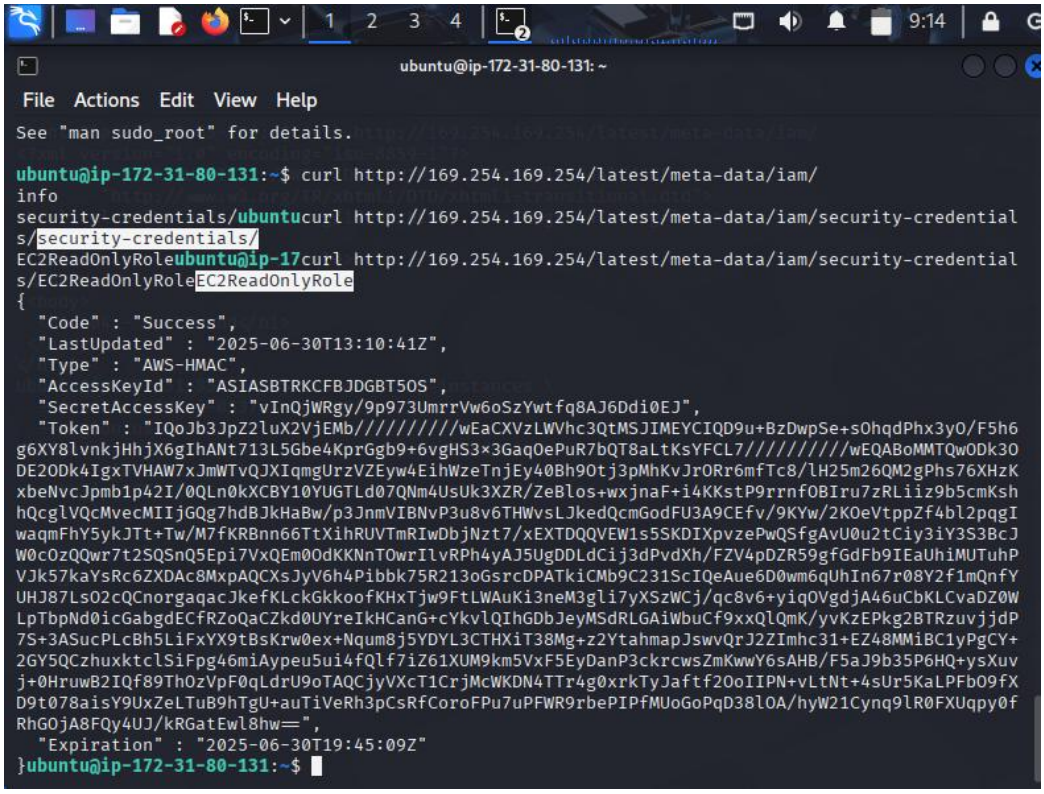


Figure 4.20: Extracting temporary AWS credentials for assigned role

Results

The test confirmed that the metadata endpoint exposed temporary AWS IAM credentials. These credentials could be harvested by any process or attacker with local access to the EC2 instance, enabling unauthorized access to AWS resources such as S3 buckets.

Analysis

This misconfiguration demonstrates a critical security risk. By default, IMDSv1 allows requests without session authentication, making it vulnerable to exploitation by malware or attackers who gain even limited access to the instance. Once credentials are obtained, they can be used outside the instance to perform actions allowed by the IAM role, such as reading S3 objects. This type of exposure has been leveraged in real world incidents, including the Capital One breach (2019), where SSRF vulnerabilities were chained with metadata access to steal sensitive data.

Mitigation

To mitigate this risk, EC2 instances should enforce IMDSv2, which requires session based authentication and mitigates many SSRF based attacks. Access to the metadata service (169.254.169.254) should be restricted using iptables or host based firewalls, preventing untrusted processes from querying it. The principle of least privilege must be applied to IAM roles, ensuring that even if temporary credentials are exposed, their scope is limited. Finally, continuous monitoring with AWS CloudTrail and Amazon GuardDuty should be enabled to detect unusual metadata queries and possible credential misuse.

5.8 Comparative Analysis of Tests

The two misconfiguration scenarios highlight distinct yet complementary risks in AWS environments.

In the S3 bucket test, the exposure was immediate and highly visible: confidential data was directly accessible without authentication. This represents a straightforward violation of data confidentiality, mapping directly to OWASP A01:2021 Broken Access Control (OWASP, 2021). Importantly, the simplicity of the exploit reflects the real world pattern in which attackers use automated tools such as Bucket Finder or S3Scanner to sweep the internet for exposed buckets, making exploitation trivial and scalable (Palo Alto Networks, 2021).

By contrast, the EC2 metadata exposure was less obvious but potentially more damaging. Through IMDSv1, attackers could harvest IAM credentials and escalate privileges across multiple services. This vulnerability aligns with OWASP A05:2021 Security Misconfiguration (OWASP, 2021), as it reflects reliance on insecure defaults. Unlike S3 exposure, which leads to immediate data leakage, EC2 metadata attacks open the door to long term compromise, persistence, and lateral movement.

When compared directly, the S3 misconfiguration is high risk in terms of data exfiltration speed, while the EC2 metadata issue is high risk in terms of attacker control. In practice, attackers frequently chain these flaws: for instance, exploiting an SSRF vulnerability in an

EC2 hosted web app to retrieve metadata credentials, then using those credentials to access exposed S3 buckets. This chained attack mirrors the 2019 Capital One breach, where misconfigured WAF rules combined with metadata exposure enabled access to millions of records (Krebs, 2019).

5.9 Discussion of Key Findings

The findings demonstrate that cloud misconfigurations are not isolated technical oversights but systemic governance issues. Both tests illustrate failures in secure defaults and least privilege enforcement.

For S3, the lesson is clear: storage should never be public by default. Although AWS has introduced “Block Public Access” controls, misconfigurations persist due to human error, legacy policies, and lack of continuous audits. This confirms findings in the Palo Alto Unit 42 Cloud Threat Report (2021), which reported that 63% of cloud storage services had public access enabled (Palo Alto Networks, 2021).

For EC2, the reliance on IMDSv1 highlights the danger of insecure defaults. AWS introduced IMDSv2 in 2019, yet adoption has lagged, leaving instances vulnerable. The Verizon DBIR (2022) underscores that such lagging adoption of security patches and best practices continues to drive breaches (Verizon, 2022). This finding suggests that technical mitigations are insufficient without strong organizational policies, automation, and cultural awareness of shared responsibility.

Another critical insight is the role of monitoring. Both tests revealed that misconfigurations are silent unless actively audited. Tools like GuardDuty and AWS Config provide detection mechanisms, but they are only effective if organizations configure alerts, review findings, and respond promptly. Misconfiguration, therefore, is less a technical gap and more a failure of governance, visibility, and accountability (AWS, 2023a).

5.10 Chapter Summary

This chapter examined two AWS misconfigurations public S3 buckets and EC2 metadata exposure through practical exploitation. Both vulnerabilities demonstrated how seemingly small configuration oversights could lead to significant security risks, including direct data leakage, unauthorized privilege escalation, and potential long term compromise.

The comparative analysis showed that while S3 exposure leads to immediate and direct data loss, EC2 metadata exploitation provides attackers with credentials that can be leveraged for broader, persistent attacks. The combination of these risks mirrors patterns seen in major breaches such as Capital One, underscoring that attackers often chain vulnerabilities for maximum impact (Krebs, 2019; UpGuard, 2017).

Ultimately, the findings emphasize three key points: (1) secure configuration defaults are essential, (2) least privilege must be enforced consistently across IAM and resource policies, and (3) continuous monitoring and detection mechanisms are necessary to address the silent, systemic nature of cloud misconfigurations (AWS, 2023a; OWASP, 2021; Verizon, 2022).

Chapter 6: Discussion and Critical Evaluation

6.1 Introduction

This chapter provides a critical reflection on the findings from the penetration testing of intentionally misconfigured AWS environments. While Chapter 4 outlined the implementation of the testbed and Chapter 5 documented the technical testing results, this section interprets those outcomes in relation to wider research, industry breach reports, and established security frameworks. The purpose is to move beyond description and assess the implications of the findings, situating them within the broader landscape of cloud security challenges.

The discussion highlights how the misconfigurations studied public S3 buckets and EC2 metadata exposure relate to known patterns of cloud compromise. These weaknesses are contextualised with reference to real world case studies, such as the Accenture (2017) and Capital One (2019) breaches, and mapped against recognised vulnerability categories in the OWASP Top Ten (2021). Furthermore, the chapter considers the legal, social, ethical, and professional issues raised by cloud penetration testing, including compliance obligations under frameworks such as GDPR and the need to follow professional codes of practice. The limitations of the research are critically evaluated, and recommendations for future work are proposed.

6.2 Interpreting the Findings

6.2.1 S3 Bucket Misconfiguration

The first test demonstrated how misconfigured S3 buckets can result in unauthorised public access to stored objects. By disabling Block Public Access, files were retrievable directly from the internet without authentication. This was evidenced during testing where a dummy file (top_secret.txt) was successfully accessed via both a web browser and curl (see Figures 5.2 and 5.3). Such exposure exemplifies OWASP Top 10 A01:2021 Broken Access Control and A05:2021 Security Misconfiguration (OWASP, 2021).

These findings mirror several high profile cases. Accenture's 2017 incident, for example, involved unsecured S3 buckets exposing sensitive API keys and internal data (UpGuard, 2017). Similarly, the U.S. Department of Defense was reported to have exposed terabytes of classified data in open Amazon storage (Vickery, 2017). The reproduction of this vulnerability in a controlled lab confirms that S3 misconfigurations remain one of the most frequent causes of cloud data exposure.

6.2.2 EC2 Metadata Service (IMDSv1) Misconfiguration

The second test revealed that EC2 instances configured to use IMDSv1 are highly susceptible to exploitation. Using curl, unauthenticated requests to the metadata endpoint

(169.254.169.254) successfully retrieved sensitive information, including the instance's public IPv4 address and temporary IAM role credentials (see Figures 5.11 and 5.12). This misconfiguration directly maps to OWASP Top 10 A05:2021 Security Misconfiguration, since insecure defaults allowed access to metadata without session authentication (OWASP, 2021).

The Capital One breach in 2019 demonstrates the severity of this vulnerability. Attackers exploited IMDSv1 through a Server-Side Request Forgery (SSRF) vulnerability to obtain AWS credentials, leading to the compromise of over 100 million customer records (Krebs, 2019). Broader industry analysis supports this risk, with Verizon's DBIR (2022) attributing a significant proportion of cloud breaches to misconfiguration, and Palo Alto Unit 42 (2021) identifying over-privileged IAM roles as a recurring weakness.

Misconfiguration Type	Evidence from Testing (Figures)	Real-World Case Example	OWASP Mapping
S3 Bucket Public Access (Read/Write)	Figure 5.2 (dummy file upload), Figure 5.3 (public retrieval via curl)	Accenture 2017 (UpGuard, 2017); DoD 2017 (Vickery, 2017)	A01 Broken Access Control; A05 Security Misconfiguration
EC2 Apache2 File Exposure	Figure 5.7 (local file access), Figure 5.8 (remote retrieval via browser)	Similar exposures in web server misconfiguration cases	A05 Security Misconfiguration
EC2 Metadata Service (IMDSv1)	Figure 5.11 (metadata categories), Figure 5.12 (retrieved public IPv4)	Capital One 2019 (Krebs, 2019)	A05 Security Misconfiguration; A07 Identification & Authentication Failures
IAM Role Exposure via Metadata	Figure 5.19 (retrieving IAM role name), Figure 5.20 (temporary credentials)	Cloud IAM role exposure trends (Palo Alto Unit 42, 2021)	A01 Broken Access Control; A05 Security Misconfiguration

Table 6.1: Summary of Misconfiguration Findings and Supporting Evidence

6.2.3 Comparative View

Taken together, the results demonstrate that S3 and EC2 misconfigurations introduce distinct yet complementary risks. In the case of S3, public bucket exposure created a direct pathway for data leakage, as evidenced by the retrieval of a dummy file without authentication (Figures 5.2 and 5.3). This mirrors incidents such as Accenture's 2017 breach, where unsecured buckets exposed credentials and internal systems (UpGuard, 2017).

By contrast, the EC2 metadata misconfiguration (Figures 5.11 and 5.12) represents a more subtle but arguably more dangerous vulnerability. While it does not immediately expose files, it enables attackers to harvest temporary IAM credentials, which can then be leveraged for privilege escalation or lateral movement. This attack path is consistent with the Capital One

2019 breach (Krebs, 2019), where IMDSv1 was exploited via SSRF to obtain credentials used for large scale data exfiltration.

When viewed side by side (Table 6.1), the distinction becomes clearer: S3 misconfigurations pose high impact but relatively straightforward risks by exposing sensitive data directly, while EC2 metadata exposures create opportunities for chained, persistent attacks. In practice, adversaries often combine these weaknesses for example, exploiting SSRF to query metadata and steal credentials, followed by S3 access to exfiltrate sensitive files (Palo Alto Networks, 2021).

The comparative findings reinforce existing industry reports (Verizon, 2022; Palo Alto Unit 42, 2021), which identify misconfiguration as the dominant vector of cloud compromise. More importantly, the results validate the need for both preventive measures (e.g., enforcing Block Public Access, enabling IMDSv2) and detective controls (e.g., GuardDuty, AWS Config) to address the dual risks of data leakage and credential theft.

6.3 Legal, Social, Ethical, and Professional Issues

6.3.1 Legal Considerations

Cloud security testing is tightly bound to legal compliance. Within the UK and EU, the General Data Protection Regulation (GDPR) and the Data Protection Act 2018 establish strict requirements around personal data handling. Misconfigured services such as S3 buckets or EC2 instances can inadvertently expose sensitive information, triggering regulatory breaches with significant penalties. This project deliberately avoided handling real user data; instead, synthetic test data (e.g., a mock confidential file) was used to comply with ethical and legal requirements. In practice, organisations must ensure that penetration testing activities are fully authorised through clear scope agreements and, where relevant, data processing impact assessments (European Union, 2016; ICO, 2018).

6.3.2 Social Considerations

The social implications of cloud misconfigurations extend beyond technical risk to matters of public trust and confidence in digital services. High profile cases, such as the Capital One breach (Krebs, 2019), demonstrate how millions of customers can be affected by poor configuration, leading to reputational damage and erosion of trust in financial institutions. By simulating misconfigurations in a controlled lab environment, this project acknowledges the societal responsibility of cybersecurity practitioners to raise awareness of risks before they manifest in production systems. More broadly, failures in security may disproportionately impact vulnerable groups, making social accountability a critical dimension of cloud security governance (Verizon, 2022).

6.3.3 Ethical Considerations

From an ethical standpoint, penetration testing must align with the principle of “do no harm.” The ACM Code of Ethics stresses the importance of honesty, integrity, and responsibility

when engaging in activities that may affect information systems (ACM, 2018). For this reason, testing in this project was strictly limited to the researcher's own AWS Free Tier environment, with no unauthorised scanning or exploitation of third party systems. All tests were scoped to avoid causing disruption, while results were documented transparently to advance academic knowledge. This approach ensures that the research contributes to cybersecurity resilience while avoiding ethical violations.

6.3.4 Professional Considerations

Professional responsibility in cybersecurity entails not only technical proficiency but also adherence to recognised standards and frameworks. The methodology in this project drew on the NIST SP 800-115 Technical Guide for Security Testing and Assessment (Scarfone et al., 2008) and the OWASP Testing Guide (OWASP, 2021), ensuring the work followed structured and reproducible processes. Professionalism also extends to clear documentation, accurate reporting, and recognition of limitations. In line with industry expectations, findings were critically analysed and linked to real world cases to bridge the gap between theory and practice.

6.3.5 Summary

Taken together, legal, social, ethical, and professional considerations shaped the design and execution of this project. Legal frameworks such as GDPR framed the boundaries of what data could be tested; social awareness underscored the real world stakes of cloud misconfigurations; ethical principles ensured testing was safe and responsible; and professional standards provided structure and credibility. Addressing these dimensions holistically strengthens the value of the research and demonstrates alignment with both academic expectations and industry practice.

6.4 Limitations of the Study

Despite providing valuable insights into the exploitation of AWS misconfigurations, this study faced several limitations that must be considered when interpreting the findings.

First, all testing was conducted within a controlled laboratory environment rather than on production systems. While this ensured ethical compliance and avoided disruption to real-world infrastructures, it also meant that the scenarios lacked some of the complexity and unpredictability found in enterprise contexts. Production environments often incorporate layered monitoring, intrusion detection, and incident response mechanisms, any of which could have influenced the feasibility and success of the attacks.

Second, the study focused on two specific misconfigurations: publicly writable S3 buckets and exposure of the EC2 Instance Metadata Service (IMDSv1). Although these vulnerabilities are among the most common and impactful reported in cloud security research (Palo Alto Networks Unit 42, 2021; Verizon, 2022), they represent only a subset of the broader AWS threat landscape. Other significant risks such as misconfigured security groups, overly permissive IAM policies, and insecure serverless deployments were excluded due to time and scope constraints. This necessarily narrows the applicability of the results.

Third, the exploitation relied primarily on manual testing using tools such as curl, SSH, and basic AWS CLI commands. While these approaches were sufficient to demonstrate proof of concept attacks, they do not reflect the breadth of vulnerabilities that could be uncovered using automated penetration testing frameworks or commercial vulnerability scanners. Consequently, the findings may underestimate the full extent of possible exploitation in practice.

Finally, the project was constrained by the limitations of the AWS Free Tier. Resource restrictions on compute, storage, and networking capacity prevented the design of large scale experiments, such as multi account setups, cross region replication, or advanced red team simulations. As a result, the testing outcomes represent small scale demonstrations rather than enterprise level compromises.

By acknowledging these limitations, the study provides transparency regarding its scope while also identifying avenues for future research. Expanding to include a wider range of misconfigurations, applying automated testing tools, and leveraging larger scale environments would provide a more comprehensive understanding of cloud security risks.

6.5 Recommendations and Future Work

The results of this project highlight actionable measures that organisations can adopt immediately, as well as opportunities for extending the scope of future academic and technical research.

6.5.1 Practical Recommendations for Organisations

The exploitation of public S3 buckets and EC2 metadata services illustrates how basic misconfigurations can have Overwhelming impacts (Verizon, 2022; Palo Alto Networks, 2021). To address these issues, organisations should implement layered controls that combine preventive, detective, and corrective mechanisms.

For S3, storage resources must adopt a “private by default” policy, where public exposure is explicitly controlled and documented. Misconfigurations are often introduced inadvertently by developers or third party integrations, which underscores the need for automated safeguards (Unit 42, 2021). AWS provides built in features such as Block Public Access settings and S3 Access Analyzer, which should be enforced at both the account and bucket levels (AWS, 2023). Regular auditing through AWS Config rules and centralised dashboards in Security Hub can ensure that changes are continuously monitored and evaluated against compliance baselines (NIST, 2008).

For EC2 instances, it is essential to enforce IMDSv2 across all environments. Unlike IMDSv1, which relies on unauthenticated HTTP requests, IMDSv2 introduces session-based tokens, mitigating risks from Server Side Request Forgery (SSRF) attacks (AWS Security Blog, 2019). Organisations should also configure host based firewalls or iptables rules to restrict access to the metadata endpoint (169.254.169.254) and apply stringent IAM role

permissions so that even if credentials are compromised, their usefulness is minimised (OWASP, 2021).

Beyond prevention, organisations must invest in detection and response. Services such as AWS GuardDuty, CloudTrail, and CloudWatch can provide visibility into unusual access patterns, such as unexpected metadata queries or anomalous S3 object reads (Amazon, 2023). Integrating these signals into a Security Information and Event Management (SIEM) platform enables real time correlation and rapid incident response (Verizon, 2022). Furthermore, adopting infrastructure as code (IaC) security scanning with tools such as Checkov or Terraform Validator allows organisations to prevent misconfigurations before deployment, shifting security left in the development lifecycle (Bridgecrew, n.d.; HashiCorp, n.d.).

A final organisational recommendation is to strengthen the human and governance dimension of cloud security. Misconfigurations frequently stem from unclear responsibility models, time pressures, or insufficient awareness among developers and administrators (CIO 2022). Clear policies, regular security training, and routine penetration tests can ensure that teams understand not only the technical aspects of cloud services but also the broader compliance and risk management context (ACM, 2018).

6.5.2 Directions for Future Research

Broader Misconfiguration Coverage
This project examined publicly writable S3 buckets and IMDSv1 metadata exposure. However, AWS environments include numerous other attack surfaces such as misconfigured Security Groups, weak IAM trust relationships, or over permissive Lambda functions (Palo Alto Networks, 2021). Exploring these areas would provide a more comprehensive map of vulnerabilities in cloud ecosystems.

Integration of Automated Testing Tools
The present study relied on manual exploitation using curl, SSH, and AWS CLI. While effective, this approach is less scalable than automated penetration testing frameworks or commercial cloud security tools. Incorporating scanners such as ScoutSuite, Prowler, or Burp Suite extensions tailored for AWS could yield more systematic findings and uncover chained attack vectors (NCC Group, n.d.; Ton, 2020).

Large Scale and Realistic Simulations
Constraints of the AWS Free Tier limited experiments to single account, small scale setups. Future work should explore multi account and multi region architectures, as well as hybrid deployments that combine on premises and cloud services (AWS, 2023). Simulating enterprise scale environments would increase the external validity of results and better reflect the challenges faced by large organisations (Verizon, 2022).

Red Team and Adversary Emulation
Expanding into red team style exercises could provide insights into how attackers chain misconfigurations with other vulnerabilities. For instance, combining SSRF exploitation with

metadata credential theft, followed by privilege escalation into S3 or RDS, would demonstrate realistic kill chains. Frameworks such as MITRE ATT&CK for Cloud can be used to structure such research (MITRE, n.d.).

Socio Technical and Policy Dimensions

Finally, there is significant scope to integrate human and organisational perspectives into cloud misconfiguration research. Misconfigurations are rarely purely technical; they often reflect gaps in governance, insufficient training, or competing business priorities (CIO (2022), 2020). Future studies could investigate how security culture, compliance obligations (e.g., GDPR, ISO 27001), and regulatory frameworks influence the prevalence of misconfigurations (European Union, 2016; ISO, 2013).

Section 6.5 Challenges and Mitigations

Several challenges arose during the project that shaped both the scope and the outcomes. First, the reliance on the AWS Free Tier restricted the scale of experiments, limiting the ability to simulate large multi-account or cross-region environments. This was mitigated by narrowing the scope to core services (S3 and EC2) and ensuring that the selected misconfigurations were representative of high-profile real-world breaches. Similar concerns have been highlighted in prior studies, which note that small-scale testbeds often lack the complexity of enterprise cloud infrastructures (Liu, 2021).

A second challenge concerned tool selection. While automated frameworks such as Pacu, Prowler, or ScoutSuite could have provided broader scanning, their integration proved problematic within the Free Tier environment. As a result, the project relied primarily on manual exploitation through AWS CLI, curl, and SSH. This choice reduced coverage but provided deeper insight into the underlying mechanisms of each vulnerability. Research supports that while automation increases breadth, manual testing can provide more detailed insight into misconfiguration root causes (Scarfone, Souppaya and Cody, 2008).

Ethical considerations also introduced constraints. To avoid breaching AWS policies or GDPR regulations, only dummy files were uploaded, and testing was strictly confined to the researcher's own account. While this ensured compliance, it inevitably reduced realism compared to testing within a live organisational environment. The importance of ethical compliance in penetration testing, especially regarding data protection laws such as the GDPR, has been emphasised in both regulatory and academic literature (European Union, 2016; Jansen and Grance, 2011).

Chapter 7: Conclusion and Future Directions

7.1 Conclusion

This dissertation set out to investigate the security risks associated with misconfigurations in Amazon Web Services (AWS), focusing specifically on publicly accessible S3 buckets and the use of the legacy EC2 Instance Metadata Service (IMDSv1). The objective was to demonstrate how such weaknesses, when left unaddressed, can compromise the confidentiality, integrity, and availability of cloud resources, and to relate these findings to real world incidents such as the Capital One breach (Krebs, 2019) and Accenture's exposed S3 buckets (UpGuard, 2017).

The research was conducted through the design and implementation of a deliberately misconfigured cloud environment using the AWS Free Tier. A controlled penetration testing methodology was adopted, drawing on established frameworks such as NIST SP 800-115 (2008) and the OWASP Testing Guide (2021), to ensure structured, ethical, and reproducible analysis. Five tests were carried out, targeting S3 and EC2 misconfigurations, and results were evaluated in relation to the OWASP Top Ten vulnerabilities.

The findings confirmed that even basic misconfigurations can have significant security implications. Public S3 buckets allowed unauthorised listing and data tampering, while EC2 metadata exposures enabled retrieval of sensitive instance details and temporary IAM credentials. These weaknesses not only violate the principle of least privilege but also provide attackers with vectors for privilege escalation and lateral movement. The comparative analysis highlighted how storage related misconfigurations present immediate data leakage risks, whereas compute based metadata flaws enable deeper, systemic compromise.

7.2 Critical Evaluation

A key strength of this project lies in its practical demonstration of cloud misconfigurations, making abstract risks more tangible. By replicating real world attack paths in a safe laboratory setting, the study contributed both to academic understanding and to practitioner awareness of common AWS pitfalls. Aligning the results with OWASP and industry reports strengthened the credibility of the findings.

However, several limitations must be acknowledged. The scope was restricted to two misconfiguration categories, leaving out others such as misconfigured security groups, over-permissive IAM trust policies, and serverless vulnerabilities (Cybersecurity Dive 2021; Verizon, 2022). The use of manual testing tools (curl, SSH, AWS CLI) provided clear demonstrations but lacked the scale and automation available through enterprise grade vulnerability scanners. Finally, reliance on the AWS Free Tier constrained the scale of experimentation, preventing exploration of multi account or cross region architectures.

Despite these limitations, the project successfully met its objectives and demonstrated the criticality of secure cloud configuration, showing that cloud misconfigurations are not simply technical oversights but systemic governance failures.

7.3 Legal, Ethical, and Professional Reflections

In addition to technical findings, this project emphasised the legal, ethical, and professional responsibilities of cloud security testing. From a legal standpoint, frameworks such as the GDPR and the UK Data Protection Act 2018 stress that misconfigured services like S3 or EC2 could lead to unlawful exposure of personal data, with severe compliance consequences (European Union, 2016; Legislation.gov.uk, 2018). By restricting experiments to dummy files within the AWS Free Tier, this project ensured no regulatory violations occurred.

Ethically, the project aligned with the ACM Code of Ethics, adhering to principles of transparency, responsibility, and "do no harm" (ACM, 2018). No unauthorised systems were tested, and all configurations were limited to the researcher's environment. Professionally, the methodology followed recognised standards including NIST SP 800-115 and the OWASP Testing Guide, ensuring reproducibility and integrity (Scarfone, Souppaya and Cody, 2008; OWASP, 2021).

By integrating these considerations alongside the technical tests (Figures 5.3 and 5.12), the project demonstrates that cloud security research must balance technical rigour with legal compliance, ethical safeguards, and professional standards. This strengthens the academic contribution of the work and ensures relevance to both industry practice and regulatory expectations.

7.4 Recommendations and Future Directions

The findings of this project highlight that simple misconfigurations can lead to severe security risks if left unaddressed. Based on the tests conducted, three priority recommendations can be drawn. First, organisations must enforce *Block Public Access* on all S3 buckets and regularly audit configurations through automated tools such as AWS Config and Security Hub (Amazon Web Services, 2023a; Amazon Web Services, 2023d). Second, EC2 instances should be configured to use IMDSv2 exclusively, supported by restrictive IAM role policies to reduce the impact of credential leakage (Amazon Web Services, 2023b; Amazon Web Services, 2023c). Finally, continuous monitoring with AWS GuardDuty and CloudTrail is essential to detect anomalous activity and provide early warning of exploitation attempts (Amazon Web Services, 2023c; Verizon, 2022).

For future research, extending the scope to include misconfigured Security Groups, serverless deployments, and cross-region architectures would provide a more complete picture of cloud security challenges (Palo Alto Networks Unit 42, 2021). Additionally, integrating automated frameworks such as ScoutSuite or Prowler could enhance scalability (NCC Group, n.d.; Ton, 2020), while red-team style simulations structured around the MITRE ATT&CK for Cloud framework would improve realism (MITRE, n.d.).

7.5 Final Reflection

Ultimately, this project demonstrated that cloud misconfigurations, though often simple in nature, represent a persistent and serious threat. The ease with which vulnerabilities were exploited in controlled experiments mirrors the patterns observed in real world breaches. The lessons drawn are clear: secure defaults must be enforced, least privilege must be embedded across policies, and continuous monitoring is essential to detect silent misconfigurations before they evolve into catastrophic breaches.

By bridging practical testing with academic frameworks, this study contributes to the growing body of research highlighting that cloud security is as much a problem of governance and awareness as it is of technology. Addressing these challenges requires ongoing collaboration between cloud providers, organisations, and the wider cybersecurity community.

Bibliography

1. Amazon Web Services (AWS) (2019). Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 Instance Metadata Service. AWS Security Blog. Available at: https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/ (Accessed: 1 September 2025).
2. Amazon Web Services (AWS) (2023a). AWS security best practices. Available at: https://docs.aws.amazon.com/security (Accessed: 17 July 2025).
3. Amazon Web Services (AWS) (2023b). AWS Well-Architected Framework – Security Pillar. Available at: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html (Accessed: 17 July 2025).
4. Amazon Web Services (AWS) (2023c). AWS IAM Best Practices. Available at: https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html (Accessed: 17 July 2025).
5. Amazon Web Services (AWS) (2023d). Amazon S3 Block Public Access – Overview. AWS Documentation. Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html (Accessed: 1 September 2025).
6. Amazon Web Services (AWS) (2023e). Security Best Practices for Amazon S3. AWS Documentation. Available at: https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html (Accessed: 1 September 2025).
7. Amazon Web Services (AWS) (2023f). Security Best Practices for Amazon EC2. AWS Documentation. Available at: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-best-practices.html (Accessed: 1 September 2025).
8. Amazon Web Services (AWS) (2023g). Use the Instance Metadata Service to access instance metadata. AWS EC2 Documentation. Available at: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html (Accessed: 1 September 2025).
9. Amazon Web Services (AWS) (2023h). AWS Command Line Interface User Guide. Available at: https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-welcome.html (Accessed: 17 July 2025).
10. Amazon Web Services (AWS) (2023i). Penetration testing. Available at: https://aws.amazon.com/security/penetration-testing/ (Accessed: 17 July 2025).
11. Amazon Web Services (AWS) (2024). AWS Free Tier FAQs. Available at: https://aws.amazon.com/free/faqs/ (Accessed: 18 July 2025).
12. Association for Computing Machinery (ACM) (2018). ACM Code of Ethics and Professional Conduct. Available at: https://www.acm.org/code-of-ethics (Accessed: 1 September 2025).

13. Bridgecrew (n.d.). Checkov: Open Source Infrastructure as Code Static Analysis. Available at: https://www.checkov.io/ (Accessed: 1 September 2025).
14. Chen, Y., Xu, Y. and Zhang, X. (2022). 'Cloud security and misconfigurations: analysis of real-world AWS incidents', Journal of Information Security, 13(4), pp. 212–228. doi: 10.4236/jis.2022.134012.
15. CIO (2022). The Top Cloud Security Threat Comes from Within. CIO. Available at: https://www.cio.com/article/416343/the-top-cloud-security-threat-comes-from-within.html (Accessed: 1 September 2025).
16. Cloud Security Alliance (2022). Top Threats to Cloud Computing: The Pandemic 11. Available at: https://cloudsecurityalliance.org/artifacts/top-threats-to-cloud-computing-pandemic-eleven/ (Accessed: 17 July 2025).
17. Cybersecurity Dive (2021). Companies missing security in rushed cloud adoptions. Available at: https://www.cybersecuritydive.com/news/cloud-threat-security-palo-alto-networks-unit-42/597851/ (Accessed: 1 September 2025).
18. Cyber Security Hub (2023). Data of over 2 million Toyota customers exposed in cloud misconfiguration. Available at: https://www.cshub.com/attacks/news/location-data-of-2-million-customers-exposed-in-toyota-data-breach (Accessed: 17 July 2025).
19. De La Fuente, T. (2023). Prowler: CIS AWS Benchmark Scanner. GitHub. Available at: https://github.com/prowler-cloud/prowler (Accessed: 18 July 2025).
20. European Union (2016). General Data Protection Regulation (GDPR), Regulation (EU) 2016/679. Available at: https://gdpr-info.eu/ (Accessed: 1 September 2025).
21. F5 Labs (2025). Campaign targets Amazon EC2 instance metadata via SSRF. Available at: https://www.f5.com/labs/articles/threat-intelligence/campaign-targets-amazon-ec2-instance-metadata-via-ssrf (Accessed: 17 July 2025).
22. Fang, L., Kumar, R. and Alqahtani, H. (2023). 'Misconfiguration detection in Infrastructure-as-Code: A review and taxonomy', Journal of Cloud Computing, 12(1), pp. 1–20.
23. Group-IB (2022). Toyota source code exposed through cloud misconfiguration. Available at: https://www.group-ib.com/blog/toyota-cloud-leak (Accessed: 8 July 2025).
24. ISO (2013). ISO/IEC 27001:2013 Information Security Management. Available at: https://www.iso.org/standard/54534.html (Accessed: 1 September 2025).
25. Jansen, W. and Grance, T. (2011). Guidelines on Security and Privacy in Public Cloud Computing (NIST SP 800-144). Gaithersburg, MD: NIST. Available at: https://csrc.nist.gov/publications/detail/sp/800-144/final (Accessed: 17 July 2025).
26. Legislation.gov.uk (2018). Data Protection Act 2018 (UK). Available at: https://www.legislation.gov.uk/ukpga/2018/12/contents (Accessed: 1 September 2025).
27. Liu, M. (2021). 'Understanding cloud misconfiguration: causes, impacts, and prevention in AWS environments', International Journal of Cloud Applications and Computing, 11(3), pp. 45–59. doi: 10.4018/IJCAC.2021070103.
28. Liu, M. (2021). 'A review of cloud service misconfiguration and its impact on enterprise security', International Journal of Cybersecurity, 8(2), pp. 34–49.

29. MITRE (n.d.). MITRE ATT&CK for Cloud Matrix. Available at: https://attack.mitre.org/matrices/enterprise/cloud/ (Accessed: 1 September 2025).
30. Mitchell, C. (2021). Exploiting the EC2 Metadata Service: A Practical Guide. Available at: https://blog.christophetd.fr/?s=ec2+instance+meta+data+service (Accessed: 17 July 2025).
31. National Institute of Standards and Technology (NIST) (2008). SP 800-115: Technical Guide to Information Security Testing and Assessment. Available at: https://csrc.nist.gov/publications/detail/sp/800-115/final (Accessed: 1 September 2025).
32. National Institute of Standards and Technology (NIST) (2022). Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1. Available at: https://www.nist.gov/cyberframework (Accessed: 8 July 2025).
33. NCC Group (n.d.). ScoutSuite: Multi-Cloud Security Auditing Tool. GitHub. Available at: https://github.com/nccgroup/ScoutSuite (Accessed: 1 September 2025).
34. Newman, L. (2019). How one hacker took down Capital One. Wired. Available at: https://www.wired.com/story/capital-one-hack-amazon-cloud-paige-thompson/ (Accessed: 8 July 2025).
35. Newman, L.H. (2019). The Alleged Capital One Hacker Didn't Cover Her Tracks. Wired, 30 July. Available at: https://www.wired.com/story/capital-one-hack-credit-card-application-data/ (Accessed: 17 July 2025).
36. Owens, M. and Newman, D. (2020). AWS penetration testing: securing cloud services. Birmingham: Packt Publishing.
37. OWASP Foundation (2020). OWASP Web Security Testing Guide v4.2. Available at: https://owasp.org/www-project-web-security-testing-guide/ (Accessed: 19 August 2025).
38. OWASP Foundation (2021). OWASP Top Ten Web Application Security Risks – 2021. Available at: https://owasp.org/Top10/ (Accessed: 1 September 2025).
39. Palo Alto Networks (2023). The state of cloud security: 2023 report. Available at: https://www.paloaltonetworks.com/resources/research/2023-cloud-security-report (Accessed: 17 July 2025).
40. Palo Alto Networks Unit 42 (2020). Cloud Threat Report: Spring 2020. Available at: https://unit42.paloaltonetworks.com/cloud-threat-report-intro/ (Accessed: 19 August 2025).
41. Palo Alto Networks Unit 42 (2021). Unit 42 Cloud Threat Report, 1H 2021. Available at: https://www.paloaltonetworks.com/resources/research/unit42-cloud-threat-report (Accessed: 1 September 2025).
42. Prowler (2023). Prowler: AWS Security Best Practices Assessment Tool. GitHub. Available at: https://github.com/prowler-cloud/prowler (Accessed: 8 July 2025).
43. Rapid7 (2022). 2022 Cloud Misconfigurations Report: Cloud Security Breaches and Attack Trends. Available at: https://www.rapid7.com/blog/post/2022/04/20/2022-cloud-misconfigurations-report-a-quick-look-at-the-latest-cloud-security-breaches-and-attack-trends/ (Accessed: 17 July 2025).

44. Rhino Security Labs (2023). Pacu: The AWS Exploitation Framework. Available at: https://rhinosecuritylabs.com/aws/pacu-open-source-aws-exploitation-framework/ (Accessed: 17 July 2025).
45. Rhino Security Labs (2023). Pacu – AWS Exploitation Framework. GitHub. Available at: https://github.com/RhinoSecurityLabs/pacu (Accessed: 8 July 2025).
46. Scarfone, K., Souppaya, M. and Cody, A. (2008). Technical Guide to Information Security Testing and Assessment (NIST SP 800-115). Gaithersburg, MD: National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-115 (Accessed: 17 July 2025).
47. TechCrunch (2023). DoD leaks sensitive emails via misconfigured cloud server. TechCrunch. Available at: https://techcrunch.com/2023/02/21/dod-email-leak-azure (Accessed: 8 July 2025).
48. Ton, L. (2020). Prowler: AWS Security Best Practices Assessment, Auditing, Hardening and Forensics Readiness Tool. GitHub. Available at: https://github.com/prowler-cloud/prowler (Accessed: 1 September 2025).
49. UpGuard (2017). System shock: How a cloud leak exposed Accenture's business. Available at: https://www.upguard.com/breaches/cloud-leak-accenture (Accessed: 19 August 2025).
50. Vickery, C. (2017). Security researcher finds classified US Army data sitting online with no password. TechCrunch. Available at: https://techcrunch.com/2017/11/28/army-nsa-inscom-aws-leak/ (Accessed: 21 August 2025).
51. Verizon (2022). 2022 Data Breach Investigations Report (DBIR). Verizon. Available at: https://www.verizon.com/business/en-gb/resources/reports/2022-data-breach-investigations-report-dbir.pdf (Accessed: 1 September 2025).